

计算机组成原理

讲师：老汤

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

第一章：计算机系统概述

第二章：总线系统

第三章：主存储器



第四章：数据的表示与运算

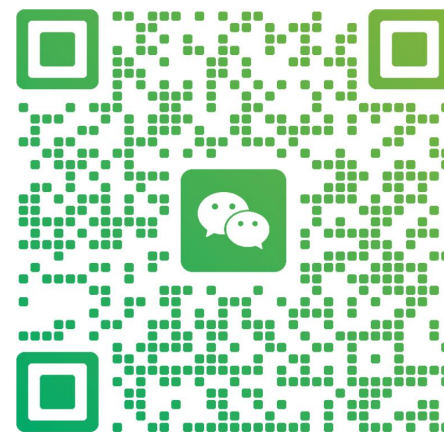
第五章：指令系统

第六章：中央处理器（CPU）

第七章：输入输出（I/O）系统

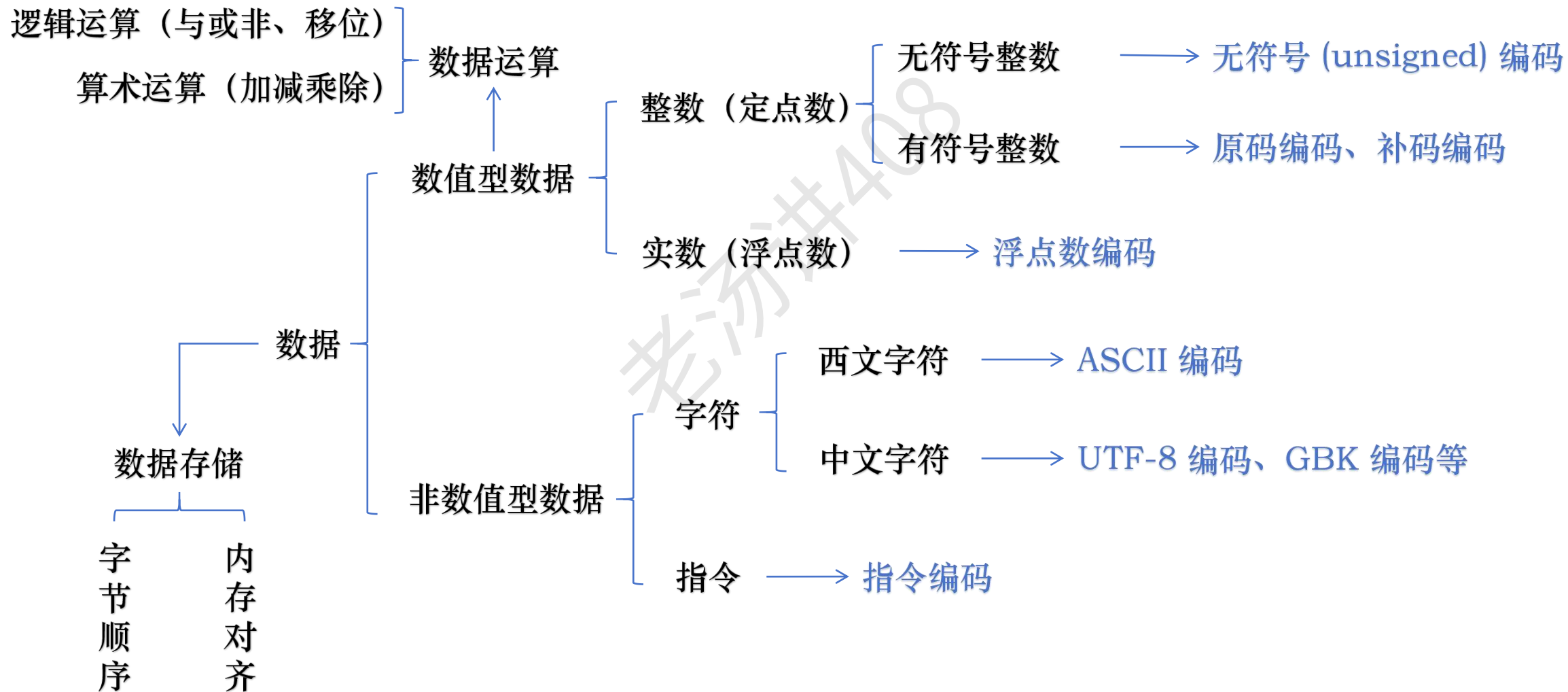


老汤
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

本章课程内容



为什么计算机系统使用二进制?

一串二进制位表示

磁盘文件

内存中的程序

内存中的用户数据

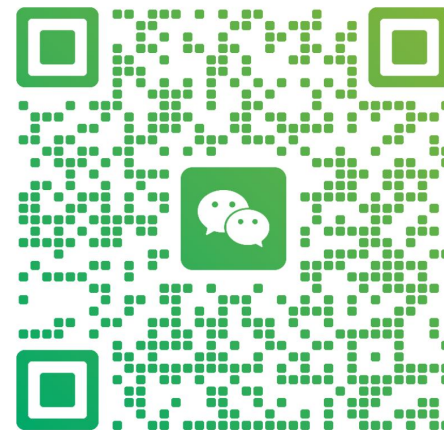
网络上传送的数据

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



老汤

浙江 杭州

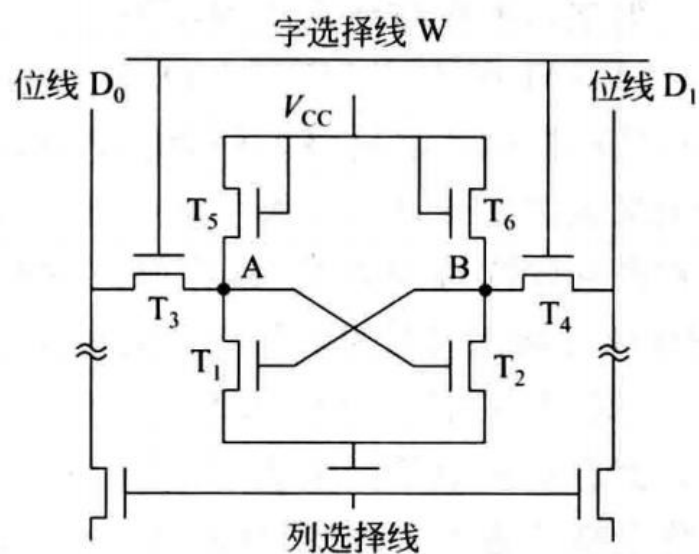


扫一扫上面的二维码图案，加我为朋友。

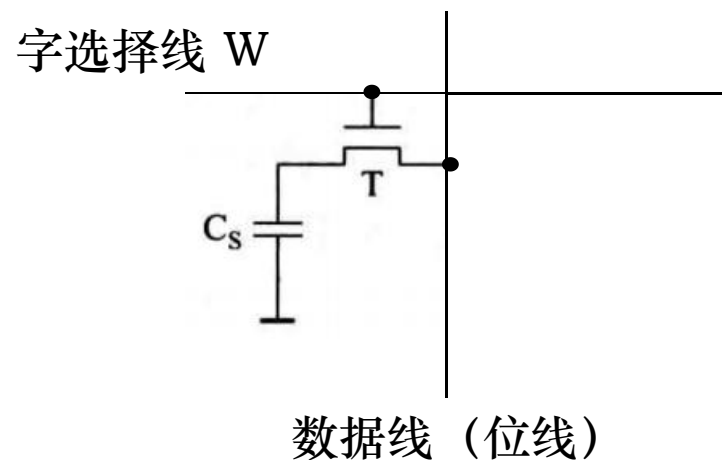
为什么计算机系统使用二进制？

① 二进制只有 0 和 1 两种基本状态，

制造有两个稳定状态的物理器件要比制造有多个稳定状态的物理器件容易得多



六管静态存储元件



单管动态存储元件

为什么计算机系统使用二进制？

- ① 二进制只有 0 和 1 两种基本状态，
制造有两个稳定状态的物理器件要比制造有多个稳定状态的物理器件容易得多
- ② 二进制的编码和运算规则都很简单，通过简单的电路就可以实现
- ③ 有一整个数学分支存在，专门处理 "真" 和 "假"，它已经证明了所有的二进制运算法则，这个分支叫做**布尔代数**，是英国科学家**乔治·布尔**提出的

1 ==> 真

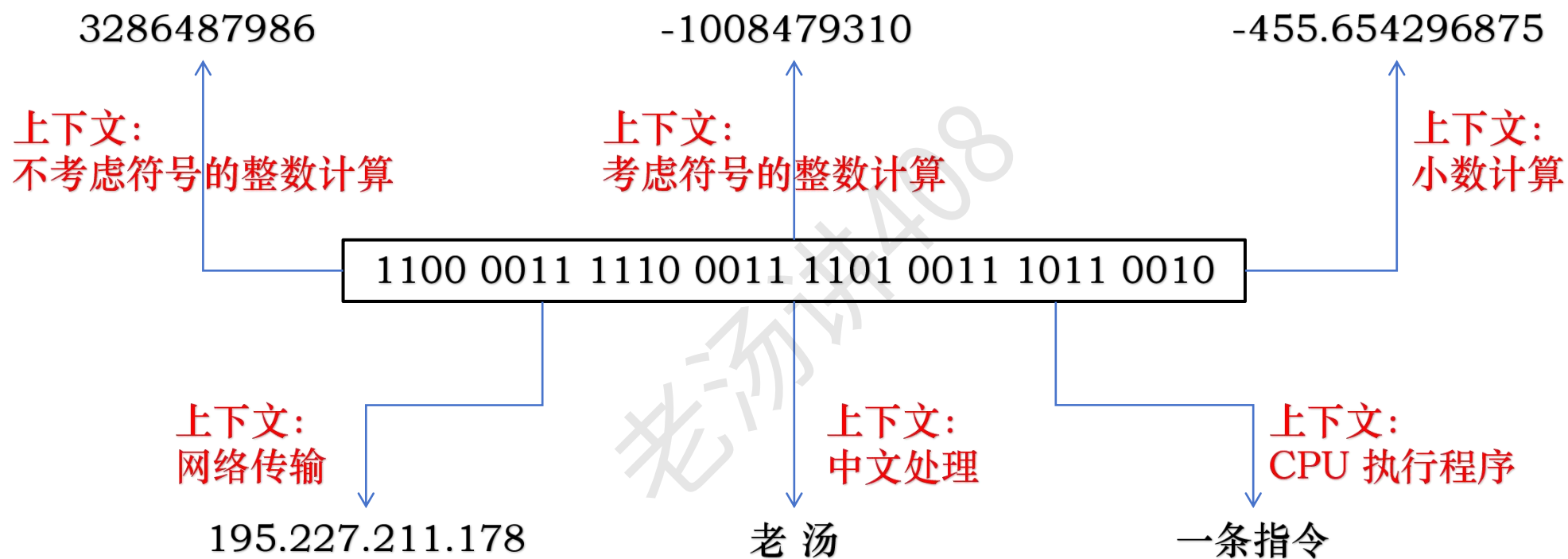
0 ==> 假

逻辑运算

程序中的逻辑判断

算术运算

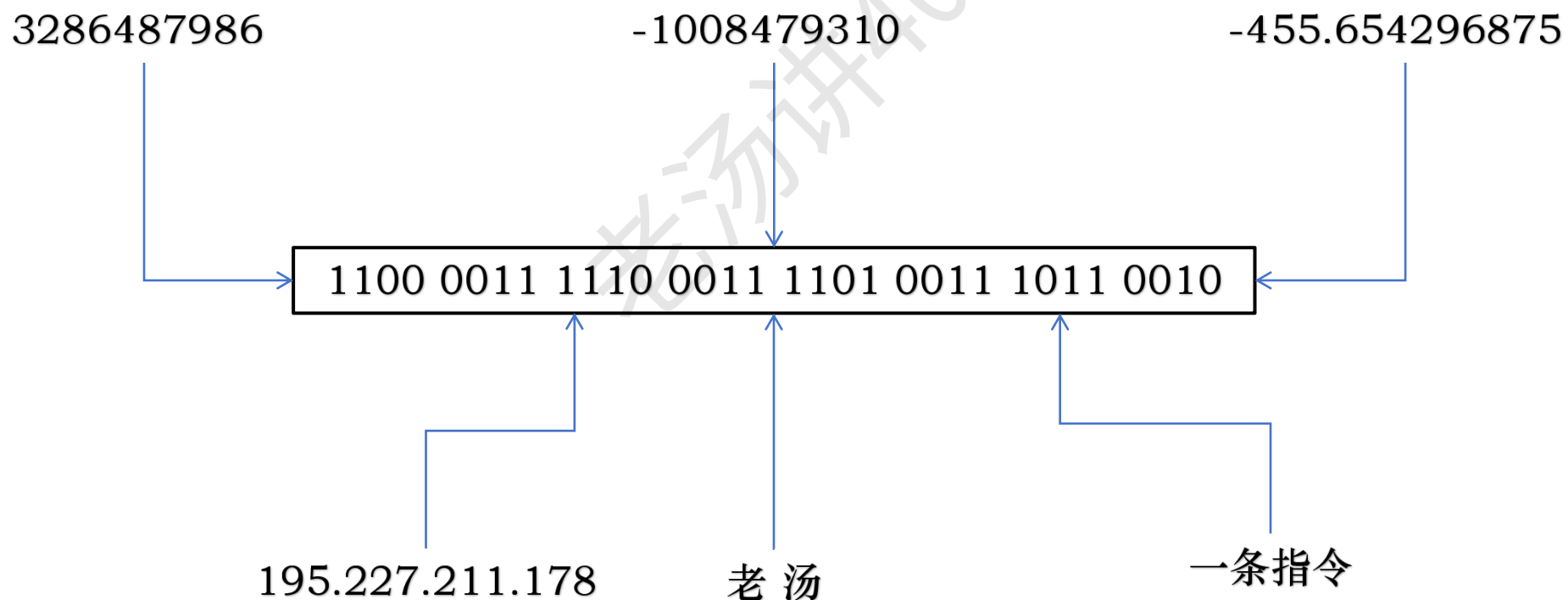
一串二进制到底表示什么信息呢？



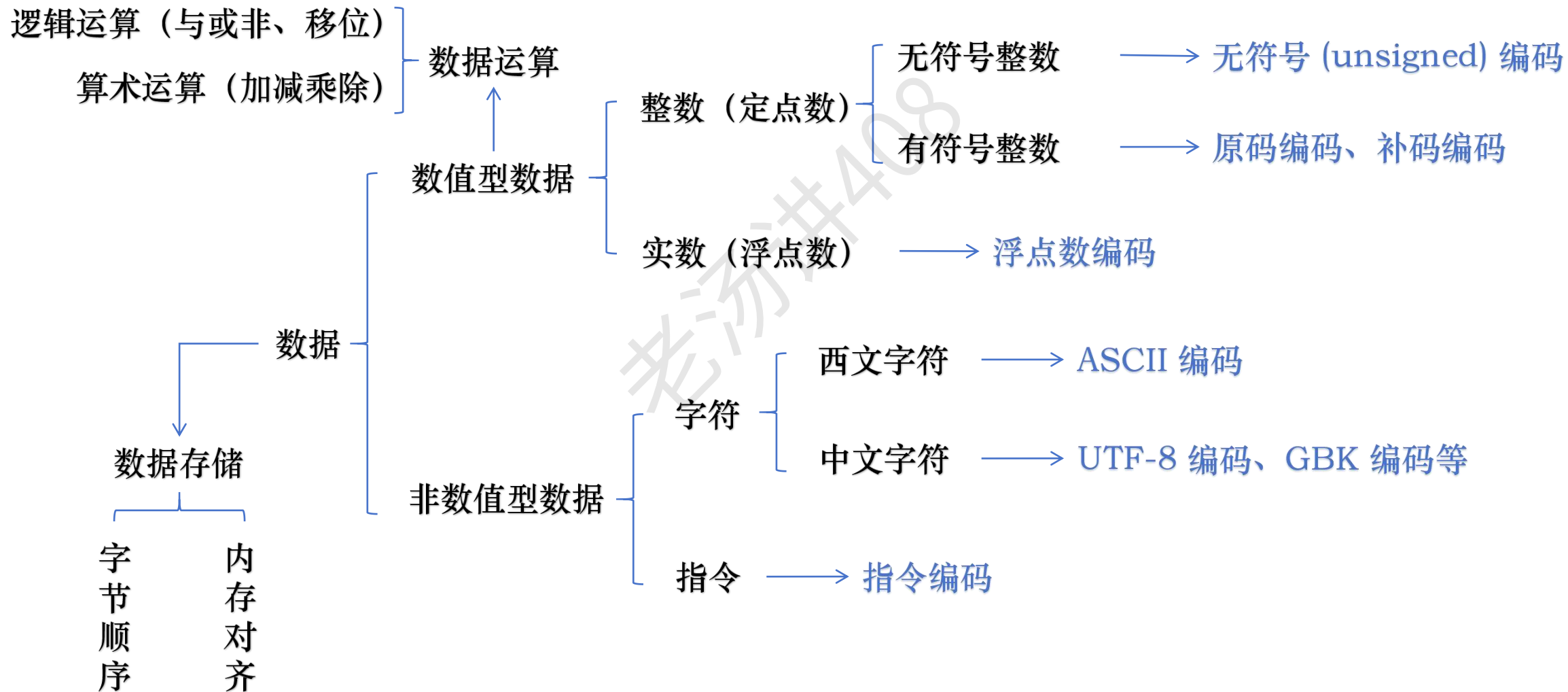
信息就是一串二进制位 + 上下文

什么是编码?

编码，就是用少量简单的基本符号对大量复杂多样的信息进行一定规则的组合



本章课程内容



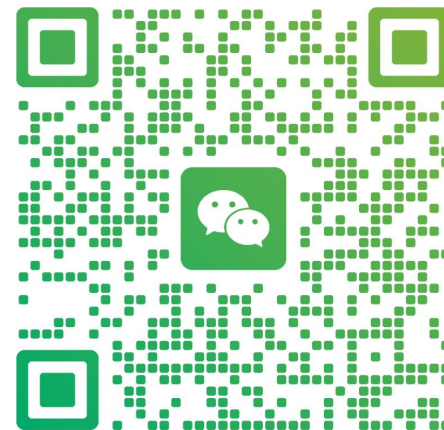
408 如何考本章的内容?

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



老汤

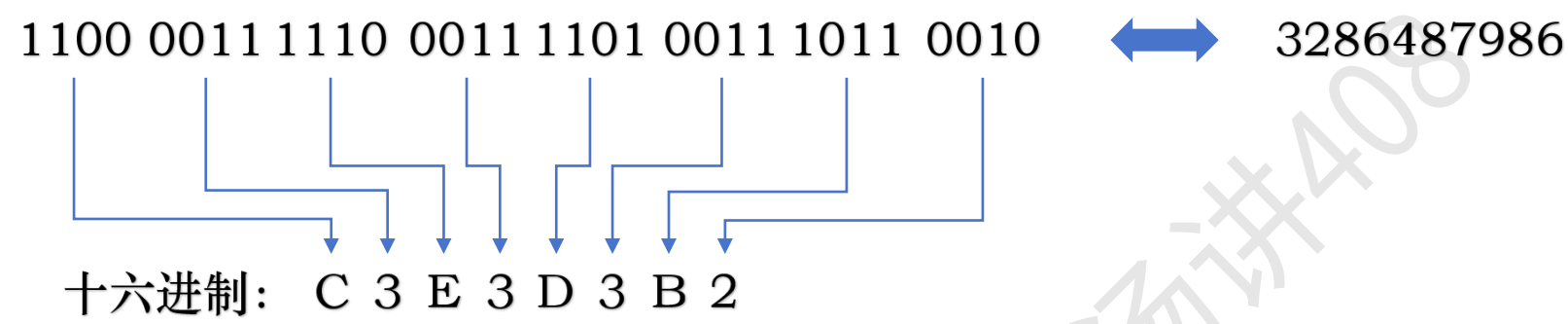
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

二进制、十进制、十六进制之间转换的技巧

- ① 二进制表示法太冗长
- ② 二进制和十进制之间转换很麻烦



C 语言中: 0xC3E3D3B2 或者 0XC3E3D3B2

C3E3D3B2H

十六进制	十进制	二进制
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

二进制、十进制、十六进制之间转换的技巧

简单的窍门：记住十六进制 A、C、F 对应的十进制和二进制

A --> 10 --> 1010
B = A + 1 B --> 11 --> 1011
C --> 12 --> 1100
D = C + 1 D --> 13 --> 1101
E = F - 1 E --> 14 --> 1110
F --> 15 --> 1111

十六进制	十进制	二进制
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
✓ A	10	1010
B	11	1011
✓ C	12	1100
D	13	1101
E	14	1110
✓ F	15	1111

二进制、十进制、十六进制之间转换的技巧

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】

练习：完成下面的数字转换

① 将 0x39A7F8 转换为二进制

0011 1001 1010 0111 1111 1000

② 将二进制 1100 1001 0111 1011 转换为十六进制

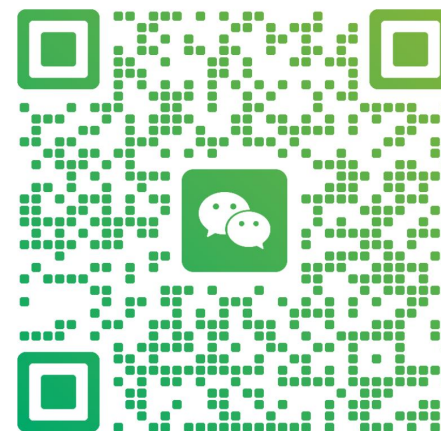
0xC97B

③ 将 0xD5E4C 转换为二进制

1101 0101 1110 0100 1100

④ 将二进制 10 0110 1110 0111 1011 0101 转换为十六进制

0x26E7B5



扫一扫上面的二维码图案，加我为朋友。

二进制、十进制、十六进制之间转换的技巧

$x = 2^n$, 其中 n 是非负数

十进制	二进制	十六进制
$1 = 2^0$	1	0x1
$2 = 2^1$	10	0x2
$4 = 2^2$	100	0x4
$8 = 2^3$	1000	0x8
$16 = 2^4$	1 0000	0x10
$32 = 2^5$	10 0000	0x20
$64 = 2^6$	100 0000	0x40
$128 = 2^7$	1000 0000	0x80
$256 = 2^8$	1 0000 0000	0x100
$512 = 2^9$	10 0000 0000	0x200
$1024 = 2^{10}$	100 0000 0000	0x400
$2048 = 2^{11}$	1000 0000 0000	0x800
$4096 = 2^{12}$	1 0000 0000 0000	0x1000

十进制	二进制	十六进制
$8192 = 2^{13}$	10 0000 0000 0000	0x2000
$16384 = 2^{14}$	100 0000 0000 0000	0x4000
$32768 = 2^{15}$	1000 0000 0000 0000	0x8000
$65536 = 2^{16}$	1 0000 0000 0000 0000	0x10000

$x = 2^n$, 二进制为 1 个 1 的后面跟 n 个 0

二进制、十进制、十六进制之间转换的技巧

$x = 2^n$, 其中 n 是非负数

十进制	二进制	十六进制
$1 = 2^0$	1	0x1
$2 = 2^1$	10	0x2
$4 = 2^2$	100	0x4
$8 = 2^3$	1000	0x8
$16 = 2^4$	1 0000	0x10
$32 = 2^5$	10 0000	0x20
$64 = 2^6$	100 0000	0x40
$128 = 2^7$	1000 0000	0x80
$256 = 2^8$	1 0000 0000	0x100
$512 = 2^9$	10 0000 0000	0x200
$1024 = 2^{10}$	100 0000 0000	0x400
$2048 = 2^{11}$	1000 0000 0000	0x800
$4096 = 2^{12}$	1 0000 0000 0000	0x1000

十进制	二进制	十六进制
$8192 = 2^{13}$	10 0000 0000 0000	0x2000
$16384 = 2^{14}$	100 0000 0000 0000	0x4000
$32768 = 2^{15}$	1000 0000 0000 0000	0x8000
$65536 = 2^{16}$	1 0000 0000 0000 0000	0x10000

$n = i + 4j$ (其中 $0 \leq i \leq 3$)

j 个 0

$0x\ 2^i\ 00..00$

$n = 11 = 3 + 4 * 2$	0x800
$n = 14 = 2 + 4 * 3$	0x4000
$n = 16 = 0 + 4 * 4$	0x10000

二进制、十进制、十六进制之间转换的技巧

$x = 2^n$, 其中 n 是非负数

十进制	二进制	十六进制	十进制	二进制	十六进制
$1 = 2^0$	1	0x1	$8192 = 2^{13}$	10 0000 0000 0000	0x2000
$2 = 2^1$	10	0x2	$16384 = 2^{14}$	100 0000 0000 0000	0x4000
$4 = 2^2$	100	0x4	$32768 = 2^{15}$	1000 0000 0000 0000	0x8000
$8 = 2^3$	1000	0x8	$65536 = 2^{16}$	1 0000 0000 0000 0000	0x10000
$16 = 2^4$	1 0000	0x10	$167 = 128 + 32 + 4 + 2 + 1$		
$32 = 2^5$	10 0000	0x20			
$64 = 2^6$	100 0000	0x40		1000 0000	
$128 = 2^7$	1000 0000	0x80		10 0000	
$256 = 2^8$	1 0000 0000	0x100			
$512 = 2^9$	10 0000 0000	0x200			
$1024 = 2^{10}$	100 0000 0000	0x400			
$2048 = 2^{11}$	1000 0000 0000	0x800			
$4096 = 2^{12}$	1 0000 0000 0000	0x1000			

二进制、十进制、十六进制之间转换的技巧

$x = 2^n$, 其中 n 是非负数

十进制	二进制	十六进制	十进制	二进制	十六进制
$1 = 2^0$	1	0x1	$8192 = 2^{13}$	10 0000 0000 0000	0x2000
$2 = 2^1$	10	0x2	$16384 = 2^{14}$	100 0000 0000 0000	0x4000
$4 = 2^2$	100	0x4	$32768 = 2^{15}$	1000 0000 0000 0000	0x8000
$8 = 2^3$	1000	0x8	$65536 = 2^{16}$	1 0000 0000 0000 0000	0x10000
$16 = 2^4$	1 0000	0x10			
$32 = 2^5$	10 0000	0x20	$167 = 128 + 32 + 4 + 2 + 1$		
$64 = 2^6$	100 0000	0x40			
$128 = 2^7$	1000 0000	0x80	$8776 = 8192 + 512 + 64 + 8$		
$256 = 2^8$	1 0000 0000	0x100	$= 2^{13} + 2^9 + 2^6 + 2^3$		
$512 = 2^9$	10 0000 0000	0x200		1 0 0 0 1 0 0 1 0 0 1 0 0 0	
$1024 = 2^{10}$	100 0000 0000	0x400			
$2048 = 2^{11}$	1000 0000 0000	0x800			
$4096 = 2^{12}$	1 0000 0000 0000	0x1000			

584 72

0 号位

二进制、十进制、十六进制之间转换的技巧

$$0x503C + 0x8 = 0x5044$$

$$\begin{array}{r} \bullet \\ 0x503C \\ + 0x8 \\ \hline 5044 \end{array}$$

$$8 + C = 8 + 12 = 20 = 4 + 16$$

二进制、十进制、十六进制之间转换的技巧

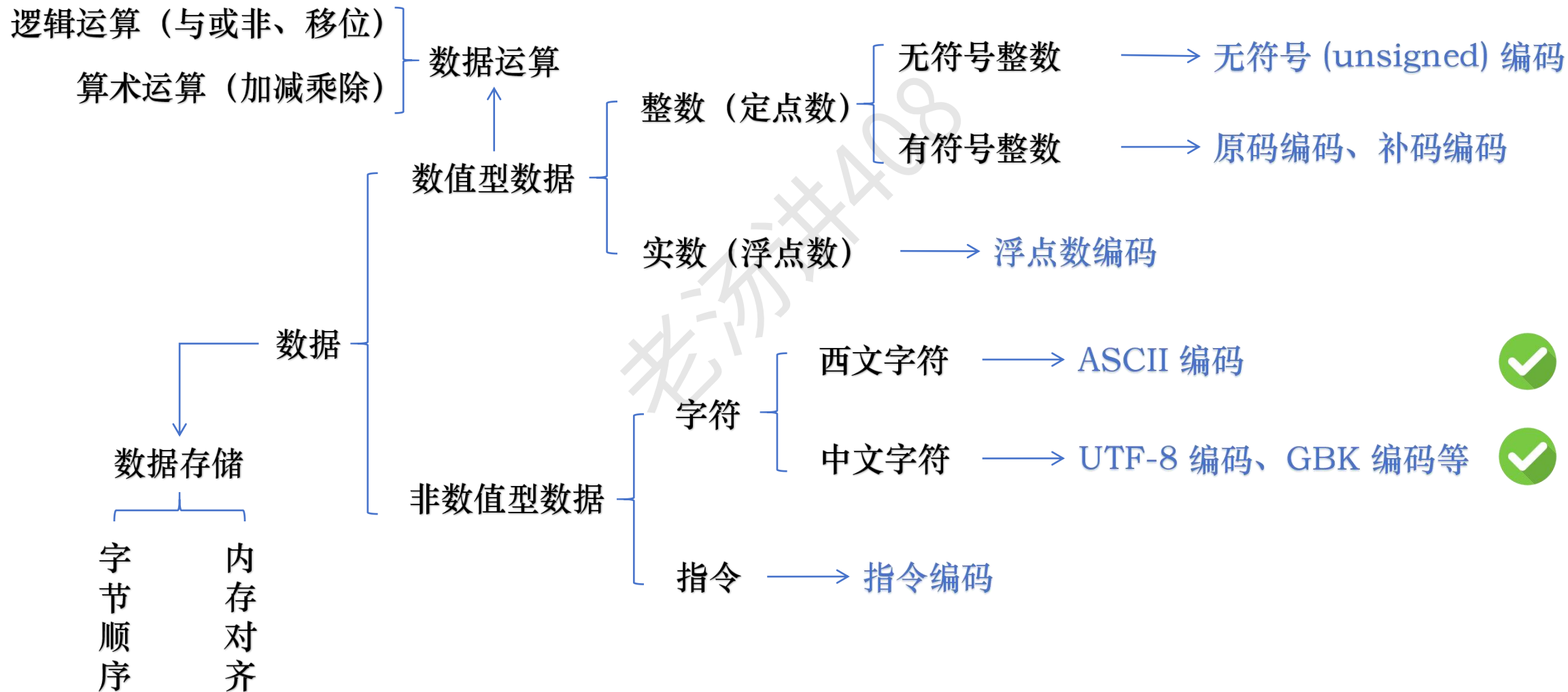
$$0x503C - 0x40 = 0x4FFC$$

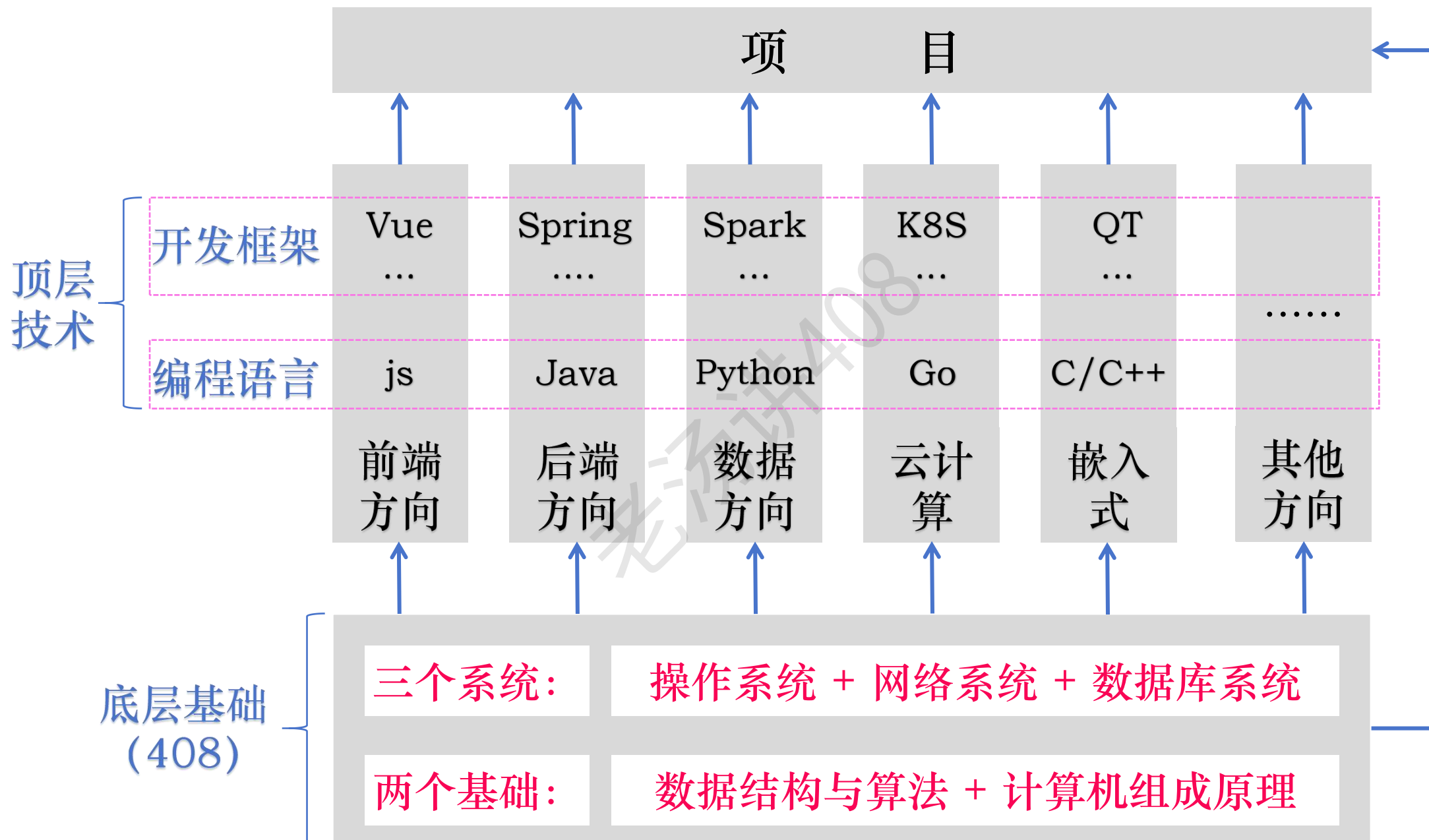
$$\begin{array}{r} \bullet \bullet \\ 0x503C \\ - \quad 0x40 \\ \hline 4FFC \end{array}$$

$$16 + 3 - 4 = 15 = F$$

$$16 - 1 = 15 = F$$

非数值数据的编码





西文字符的编码 - ASCII 编码

西文由拉丁字母、数字、标点符号及一些特殊符号所组成

ASCII: American Standard Code for Information Interchange

老汤讲408

110 0001 表示小写字母 a

011 1001 表示数字 9

`char c = 'a';`

0110 0001

有 95 个可打印的字符

有 33 个控制字符（不可打印）

```
2  #include <stdio.h>
3
4  void printBinary(char n) {
5      unsigned i;
6      for (i = 1 << 7; i > 0; i = i / 2)
7          (n & i)? printf("1"): printf("0");
8  }
9
10 int main() {
11     char c = 'a';
12     printBinary(c);
13 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\408\C\code\c_study> cd 'd:\408\C\code\c_stud
PS D:\408\C\code\c_study\ds\output> & .\'Test copy
01100001
PS D:\408\C\code\c_study\ds\output> █
```

111

p

q

r

s

t

u

v

w

x

y

z

{

|

}

~

DEL

Unicode 字符集

它为世界上几乎所有的字符都规定了一个唯一的数字编号，这个编号我们称为码点（Code Point）

英文字母“A”的 Unicode 码点是 U+0041 汉字“中”的 Unicode 码点是 U+4E2D

ㄴ U+B208	· U+0F0B	ଓ U+0CA4	🐱 U+1F639	ଠ U+09A0	└ U+0B9F	ㄟ U+4E41	ع U+063A	◦ U+26AC	🎵 U+266C	ᄁ U+104E	뽕 U+CB04	🧸 U+1F9F8	— U+4E00
ई U+0908	ঐ U+0B37	♣️ U+2663	ï U+00A1	¢ U+00A2	घ U+0918	✂️ U+2704	♠️ U+2660	ð U+00F0	← U+2190	پ U+067E	৯ U+09CE	ω U+03C9	Д U+0414
Ю U+042E	🎵 U+266B	📦 U+1F4B8	Everyone in the world should be able to use their own language on phones and computers. ▶ LEARN MORE ABOUT UNICODE  ADOPT A CHARACTER ↗										
↵ U+15D5	ℳ U+0D21	ᄎ U+0B32											
😊 U+1F60A	ÿ U+0177	ᄒ U+05D1	ᱚ U+0C20	ᱛ U+05E1	ယ U+0AAC	💛 U+1F49B	3 U+00B3	◁ U+141B	司 U+53F8	அ U+0B85	❤️ U+1F5A4	🏯 U+1F3EF	↩ U+27E8
ㄣ U+0F3D	ζ U+03B6	ꣳ U+0FD1	ঋ U+0F5D	U U+FF35	× U+00D7	£ U+00A3	🔒 U+1F512	┘ U+300D	😄 U+263B	🎯 U+1F923	🏠 U+1795	♂️ U+2642	○ U+1D3C

00000000 ~ 01111111
(0x00 ~ 0x7F)

```
00000000 00000000 ~
00000000 01111111
(0x0000 ~ 0x007F)
```

$b_6 b_5 b_4$ $b_3 b_2 b_1 b_0$	000	001	010	011	100	101	110	111
0000	NUL	DLE	SP	0	@	P	`	· p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	"	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	\$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	'	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L	\	l	
1101	CR	GS	—	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	SI	US	/	?	O	_	o	DEL

Unicode 字符集

它为世界上几乎所有的字符都规定了一个唯一的数字编号，这个编号我们称为码点 (Code Point)

英文字母 “A” 的 Unicode 码点是 U+0041 汉字 “中” 的 Unicode 码点是 U+4E2D

...00000000 01001110 00101101....



...00 4E 2D....

U+004E

U+4E2D

Unicode 字符集

它为世界上几乎所有的字符都规定了一个唯一的数字编号，这个编号我们称为码点 (Code Point)

英文字母 “A” 的 Unicode 码点是 U+0041 汉字 “中” 的 Unicode 码点是 U+4E2D

但是它没有规定这个码点在计算机中如何存储

如何在计算机中存储 Unicode 码点?

UTF-8 编码

UTF-16 编码

UTF-32 编码

**加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】**



老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

UTF-8 编码

UTF-8 最大的一个特点，就是它是一种变长的编码方式

它使用 1~4 个字节表示一个符号，根据不同的符号而变化字节长度

				0xE4B8A5
严	U+4E25	01001110 00100101	11100100 10111000 10100101	
Á	U+00C1	00000000 11000001	11000011 10000001	0xC381
A	U+0041	00000000 01000001	01000001	0x41

Unicode 码点范围

UTF-8 编码方式

00000000 00000000 ~ 00000000 01111111	0x0000 ~ 0x007F	0xxxxxxx
00000000 10000000 ~ 00000111 11111111	0x0080 ~ 0x07FF	110xxxxx 10xxxxxx
00001000 00000000 ~ 11111111 11111111	0x0800 ~ 0xFFFF	1110xxxx 10xxxxxx 10xxxxxx
	0x010000 ~ 0x10FFFF	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx
0000 0001 00000000 00000000 ~ 0001 0000 11111111 11111111		

UTF-8 编码

Unicode 码点范围

严 U+4E25 01001110 00100101

UTF-8 编码方式

11100100 10111000 10100101

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <wchar.h>
5  #include <locale.h>
6
7  void printUTF_8(wchar_t wstr[], int n) {
8      for (int i = 0; i < n; i++) {
9          for (int j = 7; j >= 0; j--)
10             printf("%d", (wstr[i] >> j) & 1);
11         printf(" ");
12     }
13 }
14
15 int main() {
16     // 存储汉字的字符串
17     char str[] = "严";
18     // 宽字符数组
19     wchar_t wstr[3];
20     // 将多字节字符串转换为宽字符字符串
21     mbstowcs(wstr, str, strlen(str) + 1);
22     // 打印宽字符的二进制表示
23     printUTF_8(wstr, 3);
24     return 0;
25 }
```

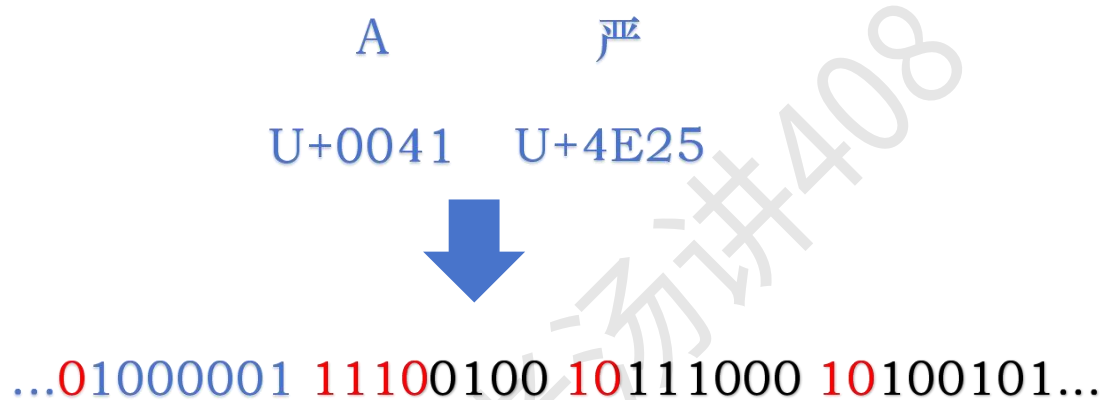
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\408\C\code\c_study\ds\output> cd 'd:\408\C\code'
PS D:\408\C\code\c_study\ds\output> & .\'UTF_8.exe'
11100100 10111000 10100101
```

UTF-8 编码

UTF-8 最大的一个特点，就是它是一种变长的编码方式

它使用 1~4 个字节表示一个符号，根据不同的符号而变化字节长度



GBK 编码

GB2312

GBK 采用双字节编码方案。对于简体中文汉字，通常使用两个字节来表示一个汉字

对于 ASCII 字符（英文字符、数字、常见符号等），GBK 采用单字节编码
其编码范围与 ASCII 编码相同，即 0x00 到 0x7F

对于汉字，GBK 的编码范围是从 0x8140 到 0xFEFE

GBK 编码

严: 0xD1CF 11010001 11001111

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <wchar.h>
5  #include <locale.h>
6
7  void printGBK(wchar_t wstr[], int n) {
8      for (int i = 0; i < n; i++) {
9          for (int j = 7; j >= 0; j--)
10             printf("%d", (wstr[i] >> j) & 1);
11         printf(" ");
12     }
13 }
14
15 int main() {
16     // 存储汉字的字符串
17     char str[] = "严";
18     // 宽字符数组
19     wchar_t wstr[2];
20     // 将多字节字符串转换为宽字符串
21     mbstowcs(wstr, str, strlen(str) + 1);
22     // 打印宽字符的二进制表示
23     printGBK(wstr, 2);
24     return 0;
25 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS D:\408\C\code\c_study\ds\output> cd 'd:\408\C\code\'

PS D:\408\C\code\c_study\ds\output> & .\'GBK.exe'

11010001 11001111

PS D:\408\C\code\c_study\ds\output> █

乱码问题

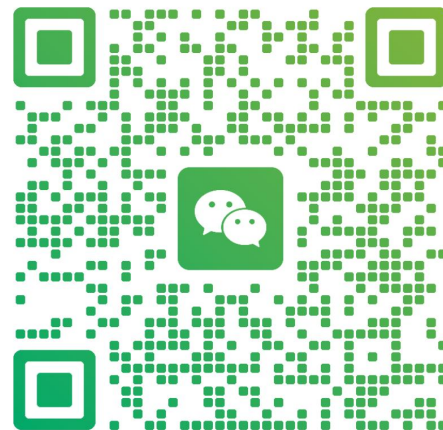
编码不一致

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



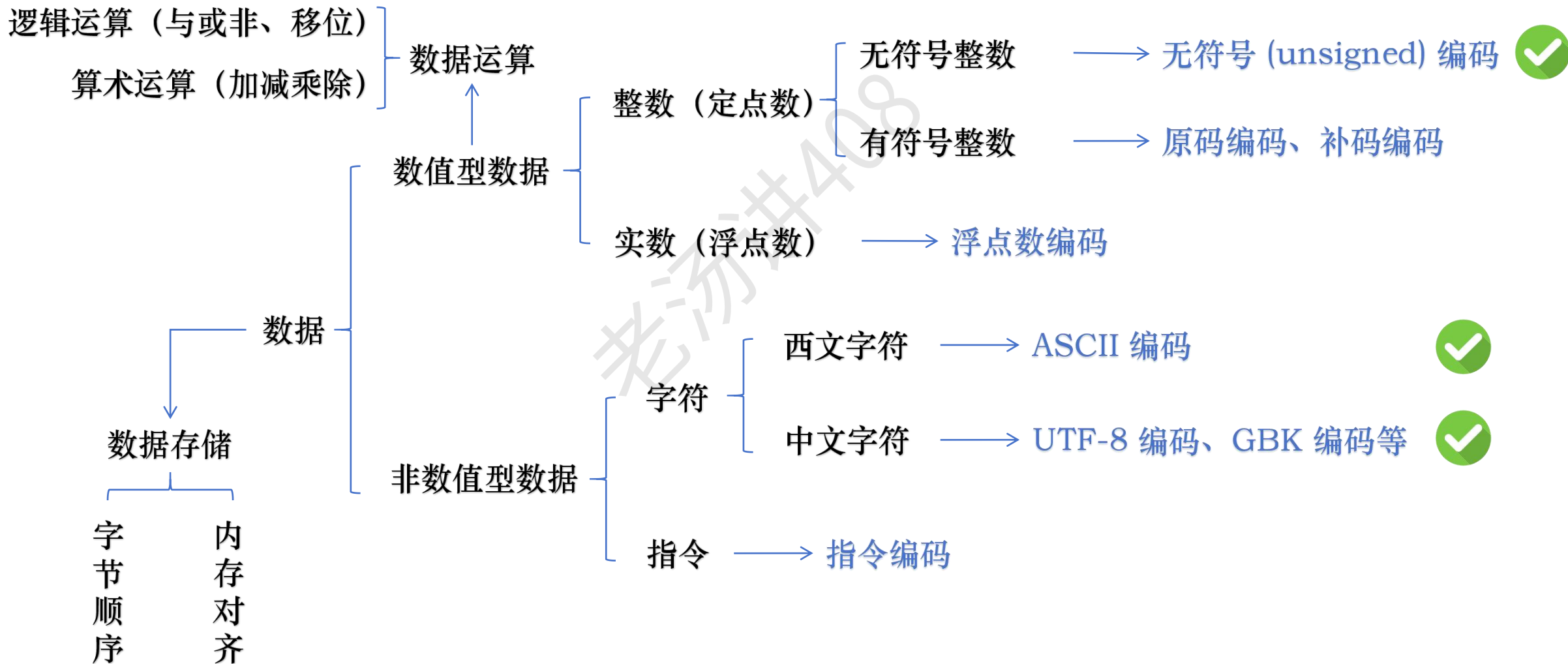
老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

数值型数据的编码



无符号 (unsigned) 整数的编码

当一个编码的所有二进制位都用来表示数值，而没有符号位时，该编码表示的就是无符号整数

$$10010010 = 1 * 2^7 + 0 * 2^6 + 0 * 2^5 + 1 * 2^4 + 0 * 2^3 + 0 * 2^2 + 1 * 2^1 + 0 * 2^0 = 146$$

$$n \text{ 位二进制: } x_{n-1}x_{n-2}\dots x_2x_1x_0 = x_{n-1} \times 2^{n-1} + x_{n-2} \times 2^{n-2} + \dots + x_2 \times 2^2 + x_1 \times 2^1 + x_0 \times 2^0 = \sum_{i=0}^{n-1} x_i 2^i$$

无符号整数就是**正整数**或者说**非负整数**

无符号 (unsigned) 整数的编码

最小无符号整数:

0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---

 0

最大无符号整数:

1	1	1	1	1	1	1	1
---	---	---	---	---	---	---	---

 $2^8 - 1 = 255$

+

1

1	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---

 $2^8 = 256$

8 位二进制位表示的无符号数的范围: $0 \sim 2^8 - 1$ ($0 \sim 255$)

16 位二进制位表示的无符号数的范围: $0 \sim 2^{16} - 1$ ($0 \sim 65535$)

32 位二进制位表示的无符号数的范围: $0 \sim 2^{32} - 1$ ($0 \sim 4294967295$)

n 位二进制位表示的无符号数的范围: $0 \sim 2^n - 1$

C 语言中的无符号 (unsigned) 整数

// 无符号字符类型, 1 字节 (0 ~ 255)

unsigned char c = 'A';

// 无符号短整型, 2 字节 (0 ~ 65535)

unsigned short s = 289;

// 无符号整型, 4 字节 (0 ~ 4294967295)

unsigned int i = 36678;

// 无符号长整型, 4 字节或者 8 字节

unsigned long l = 655898;

32 位程序

64 位程序

gcc -m32 data_type.c -o data_type_64

```
int main() {  
    unsigned long l = 234567;  
    return 0;  
}
```

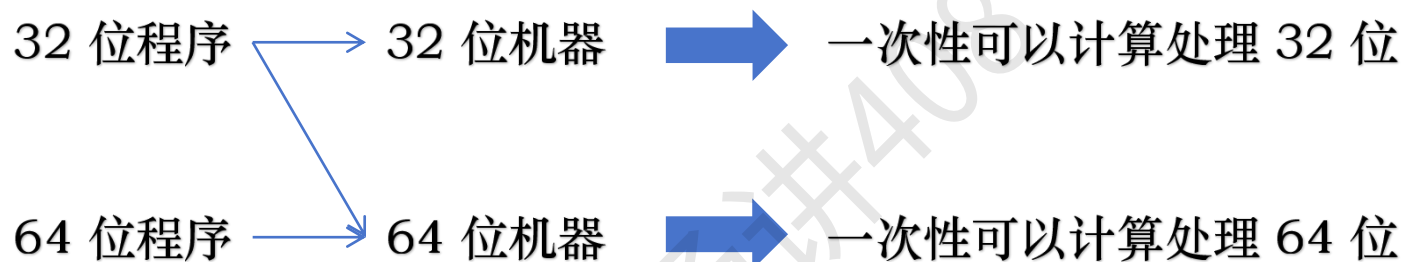
gcc -m64 data_type.c -o data_type_64

```
.cfi_startproc  
pushl    %ebp  
.cfi_def_cfa_offset 8  
.cfi_offset 5, -8  
movl     %esp, %ebp  
.cfi_def_cfa_register 5  
subl     $16, %esp  
movl     $234567, -4(%ebp)  
movl     $0, %eax  
leave  
.cfi_restore 5  
.cfi_def_cfa 4, 4  
ret
```

```
.cfi_startproc  
pushq    %rbp  
.cfi_def_cfa_offset 16  
.cfi_offset 6, -16  
movq     %rsp, %rbp  
.cfi_def_cfa_register 6  
movq     $234567, -8(%rbp)  
movl     $0, %eax  
popq     %rbp  
.cfi_def_cfa 7, 8  
ret
```

32 位机器 vs 64 位机器

机器字长就是这台机器的 CPU 一次性可以计算处理多少位的数据



按下 `Win + R` 键打开“运行”对话框。

输入 `msinfo32` 并回车，这将打开“系统信息”窗口。

在“系统信息”窗口中，找到“系统类型”一项，会显示类似于“基于 x64 的电脑”或“基于 x86 的电脑”的信息。“基于 x64 的电脑”表示你的电脑是 64 位的，“基于 x86 的电脑”表示你的电脑是 32 位的。

32 位机器 vs 64 位机器

C 声明类型	32 位程序	64 位程序
unsigned char	1 字节	1 字节
unsigned short	2 字节	2 字节
unsigned int	4 字节	4 字节
unsigned long	4 字节	8 字节

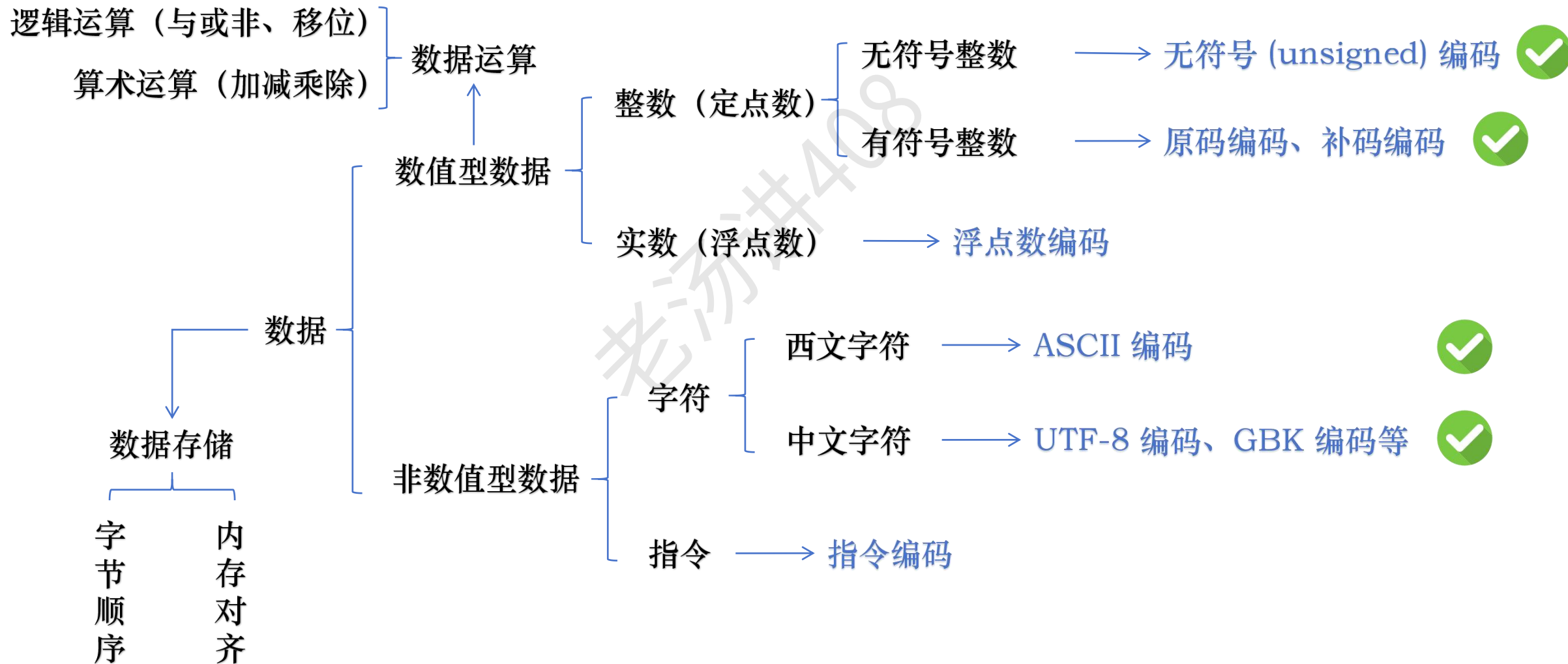
无符号数的加法

机器字长 = 8 位

	1	1	0	1	1	1	0	1	221	
+	1	1	0	0	0	0	0	1	193	
	1	1	0	0	1	1	1	1	0	158

溢出

数值型数据的编码



有符号整数的编码

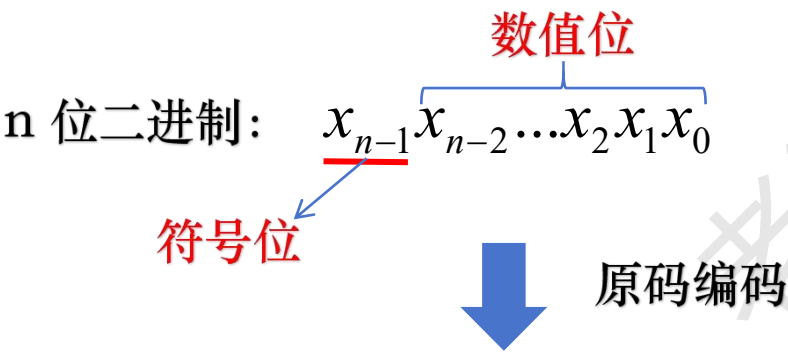
机器字长 = 4 位

无符号数编码

有符号数

n 位二进制: $x_{n-1}x_{n-2}\dots x_2x_1x_0$

无符号数: $x_{n-1} \times 2^{n-1} + x_{n-2} \times 2^{n-2} + \dots + x_2 \times 2^2 + x_1 \times 2^1 + x_0 \times 2^0 = \sum_{i=0}^{n-1} x_i 2^i$



$(-1)^{x_{n-1}} * (x_{n-2} * 2^{n-2} + \dots + x_2 * 2^2 + x_1 * 2^1 + x_0 * 2^0) = (-1)^{x_{n-1}} * (\sum_{i=0}^{n-2} x_i * 2^i)$

$0101 = (-1)^0 * (1 * 2^2 + 0 * 2^1 + 1 * 2^0) = 5$

$1101 = (-1)^1 * (1 * 2^2 + 0 * 2^1 + 1 * 2^0) = -5$

二进制	十进制
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1001	9
1010	10
1011	11
1100	12
1101	13
1110	14
1111	15

二进制	十进制
0000	+0
0001	+1
0010	+2
0011	+3
0100	+4
0101	+5
0110	+6
0111	+7
1000	-0
1001	-1
1010	-2
1011	-3
1100	-4
1101	-5
1110	-6
1111	-7

有符号整数的编码：原码

有符号数 -> 原码 机器字长 = 8 位

6 原码

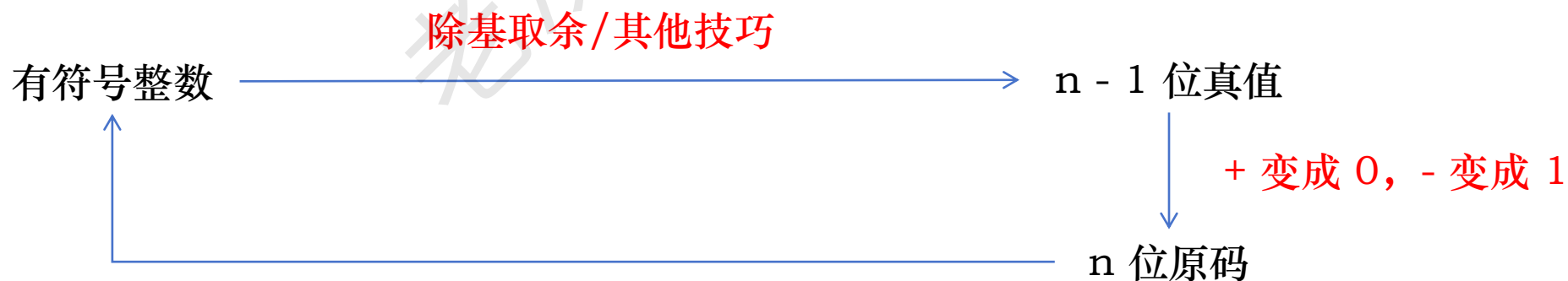
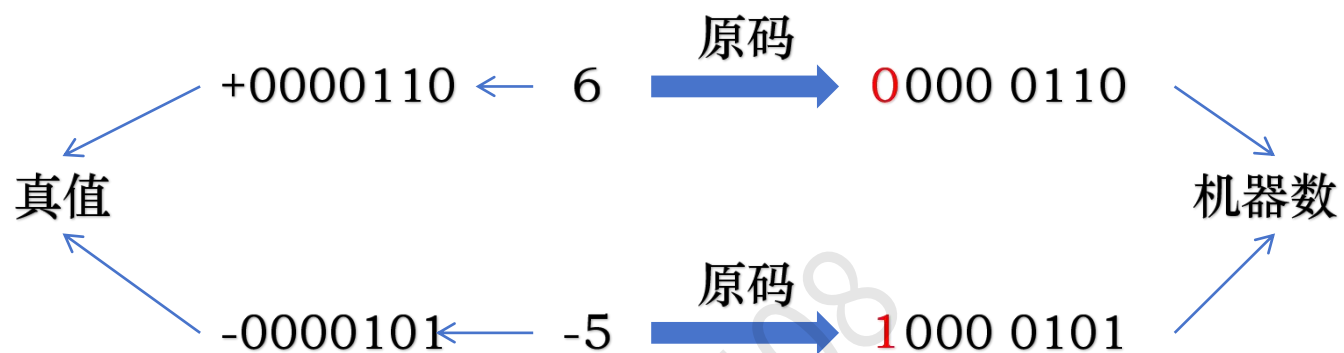
0	0	0	0	0	1	1	0
---	---	---	---	---	---	---	---

-5 原码

1	0	0	0	0	1	0	1
---	---	---	---	---	---	---	---

有符号整数的编码：原码

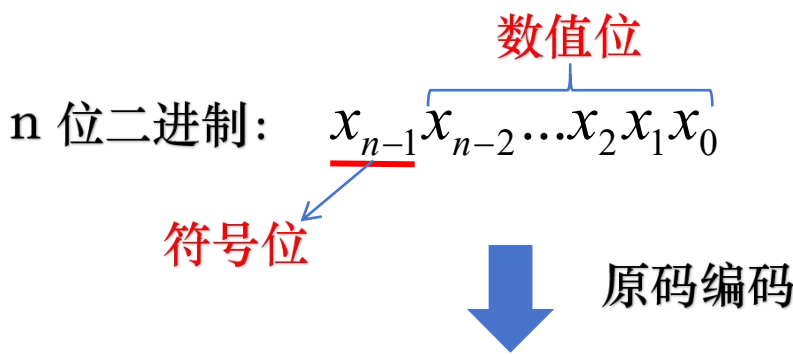
机器字长 = 8 位



$$(-1)^{x_{n-1}} * (x_{n-2} * 2^{n-2} + \dots + x_2 * 2^2 + x_1 * 2^1 + x_0 * 2^0) = (-1)^{x_{n-1}} * \left(\sum_{i=0}^{n-2} x_i * 2^i \right)$$

有符号整数的编码（原码）

机器字长 = 4 位 原码编码



$$(-1)^{x_{n-1}} * (x_{n-2} * 2^{n-2} + \dots + x_2 * 2^2 + x_1 * 2^1 + x_0 * 2^0) = (-1)^{x_{n-1}} * (\sum_{i=0}^{n-2} x_i * 2^i)$$

最小值: $\overbrace{111111\dots1}^{n-1} = -(2^{n-1} - 1)$

最大值: $\overbrace{011111\dots1}^{n-1} = 2^{n-1} - 1$

n 位原码表示有符号整数的范围: $-(2^{n-1} - 1) \sim 2^{n-1} - 1$

二进制	十进制
0000	+0
0001	+1
0010	+2
0011	+3
0100	+4
0101	+5
0110	+6
0111	+7
1000	-0
1001	-1
1010	-2
1011	-3
1100	-4
1101	-5
1110	-6
1111	-7

有符号整数的编码：原码

优点：

① 和真值之间的转换很简单

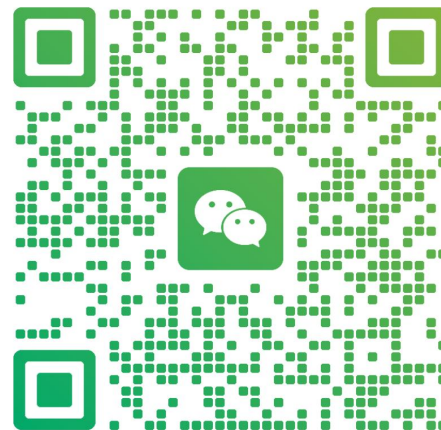
② 用原码实现乘除运算比较简便

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

有符号整数的编码：原码（0 的编码不唯一）

n 位二进制： $\underbrace{x_{n-1}}_{\text{符号位}} \overbrace{x_{n-2} \dots x_2 x_1 x_0}^{\text{数值位}}$



原码编码

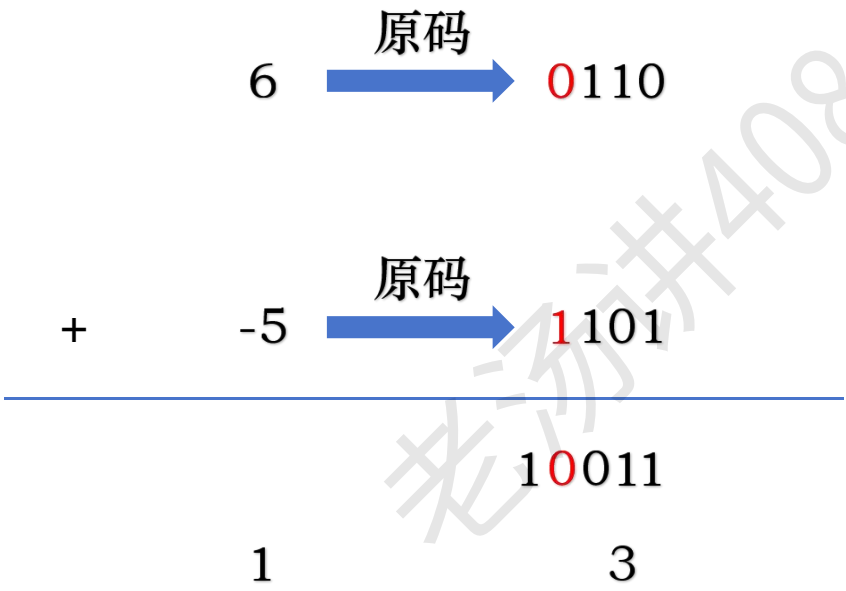
$$(-1)^{x_{n-1}} * (x_{n-2} * 2^{n-2} + \dots + x_2 * 2^2 + x_1 * 2^1 + x_0 * 2^0) = (-1)^{x_{n-1}} * \left(\sum_{i=0}^{n-2} x_i * 2^i \right)$$

1000...0 $\longleftrightarrow$ -0

0000...0 $\longleftrightarrow$ +0

有符号整数的编码：原码（加减运算规则复杂）

机器字长 = 4 位 原码编码



原码的最高位是符号位，所以不能直接参与加减运算

二进制	十进制
0000	+0
0001	+1
0010	+2
0011	+3
0100	+4
0101	+5
0110	+6
0111	+7
1000	-0
1001	-1
1010	-2
1011	-3
1100	-4
1101	-5
1110	-6
1111	-7

有符号整数的编码（什么是补码？）

$$\begin{array}{r} 253 \rightarrow \text{被减数} \\ - 176 \rightarrow \text{减数} \\ \hline 77 \end{array}$$

数字电路实现减法时比较困难的，或者说实现起来，成本比较高

因为减法中存在**借位**，就是这个借位实现起来比较麻烦

有符号整数的编码（什么是补码？）

补数 从一串 9 中减去一个数叫做对 9 求补数 (Nines' complement)

2 5 3 → 被减数	9 9 9	2 5 3	1 0 7 6
- 1 7 6 → 减数	- 1 7 6	+ 8 2 3	+ 1
7 7	8 2 3	1 0 7 6	1 0 7 7
			- 1 0 0 0
			7 7

-176 对 9 的补数是 823

-823 对 9 的补数是 176

有符号整数的编码（什么是补码？）

$\begin{array}{r} 253 \rightarrow \text{被减数} \\ - 176 \rightarrow \text{减数} \\ \hline 77 \end{array}$	$\begin{array}{r} 999 \\ - 176 \\ \hline 823 \end{array}$	$\begin{array}{r} 253 \\ + 823 \\ \hline 1076 \end{array}$	$\begin{array}{r} 1076 \\ + 1 \\ \hline 1077 \\ - 1000 \\ \hline 77 \end{array}$
└──┘			

$1000 - 176 = 824$

10 的补数 (Ten' s complement)

-176 对 10 的补数是 824

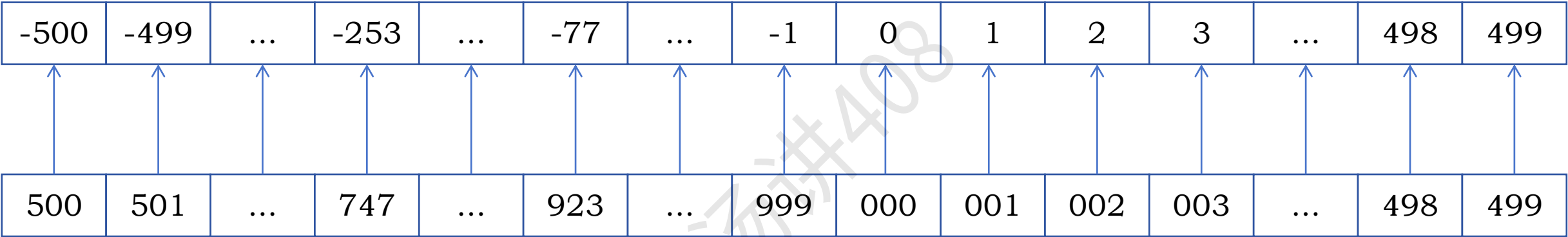
$999 - 176 + 253 + 1 - 1000$



$253 - 176 + 1000 - 1000$

有符号整数的编码（什么是补码？）

3 个十进制数



对 10 的补数 (Ten' s complement)

$$\begin{array}{r} 176 \\ - 253 \\ \hline -77 \end{array}$$

$$\begin{array}{r} 176 \\ + 747 \\ \hline 923 \end{array}$$

有符号整数的编码 (什么是补码?)

机器字长 = 4 位

对 1 的补数 (ones' complement)

在二进制中求对 1 的补码不需要做减法，只需要对这里的数进行取反即可

$$\begin{array}{r} 6 \\ - 3 \\ \hline \end{array}$$

$$\begin{array}{r} 0110 \\ - 0011 \\ \hline \end{array}$$

$$\begin{array}{r} 1111 \\ - 0011 \\ \hline 1100 \\ + 1 \\ \hline 1101 \end{array}$$

$$\begin{array}{r} 0110 \\ + 1101 \\ \hline 0011 \\ - 10000 \\ \hline \end{array} = 1111 + 1$$

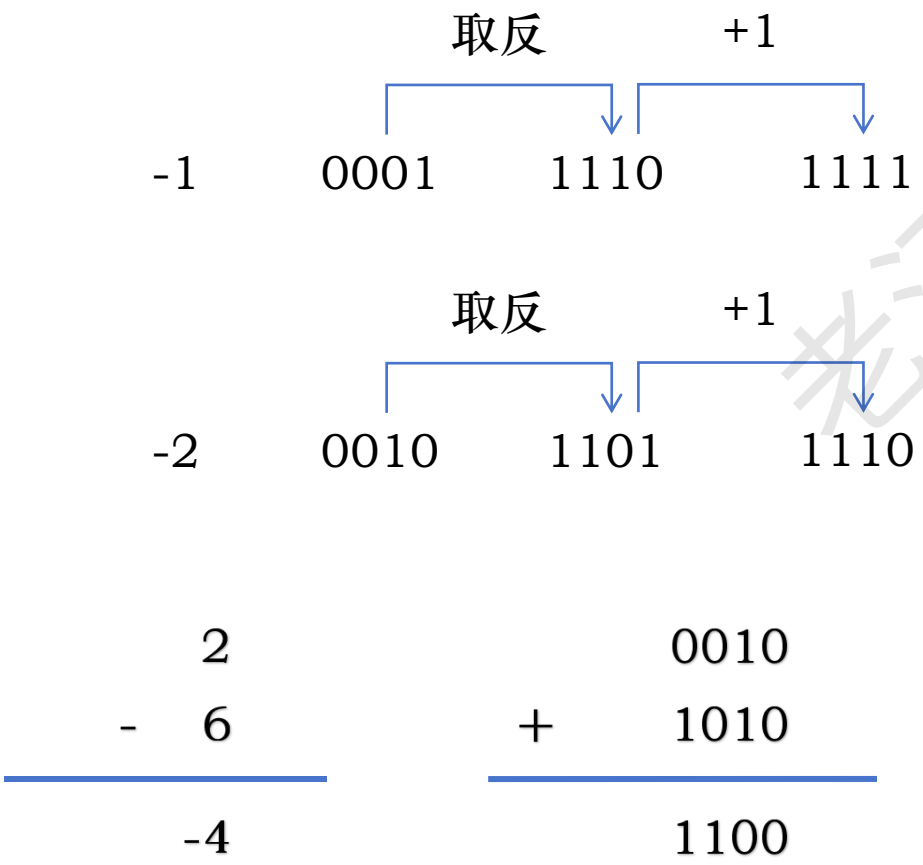
-3 对 2 的补数 (Two' s complement) 补码

减去一个数，相当于加上这个数的补码

而这个数的补码通过对这个数进行取反再加 1 得到

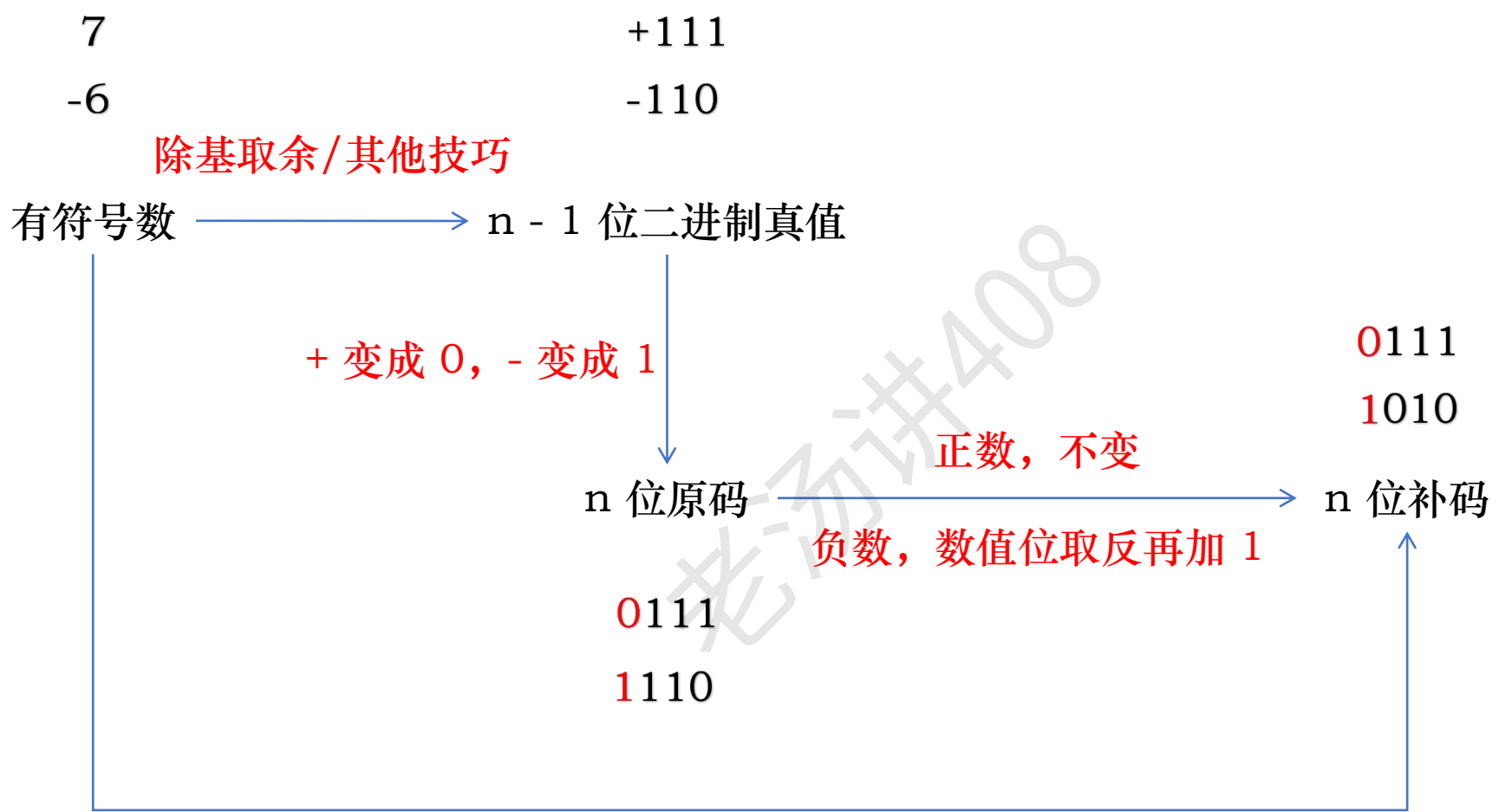
有符号整数的编码：补码

4 个二进制数



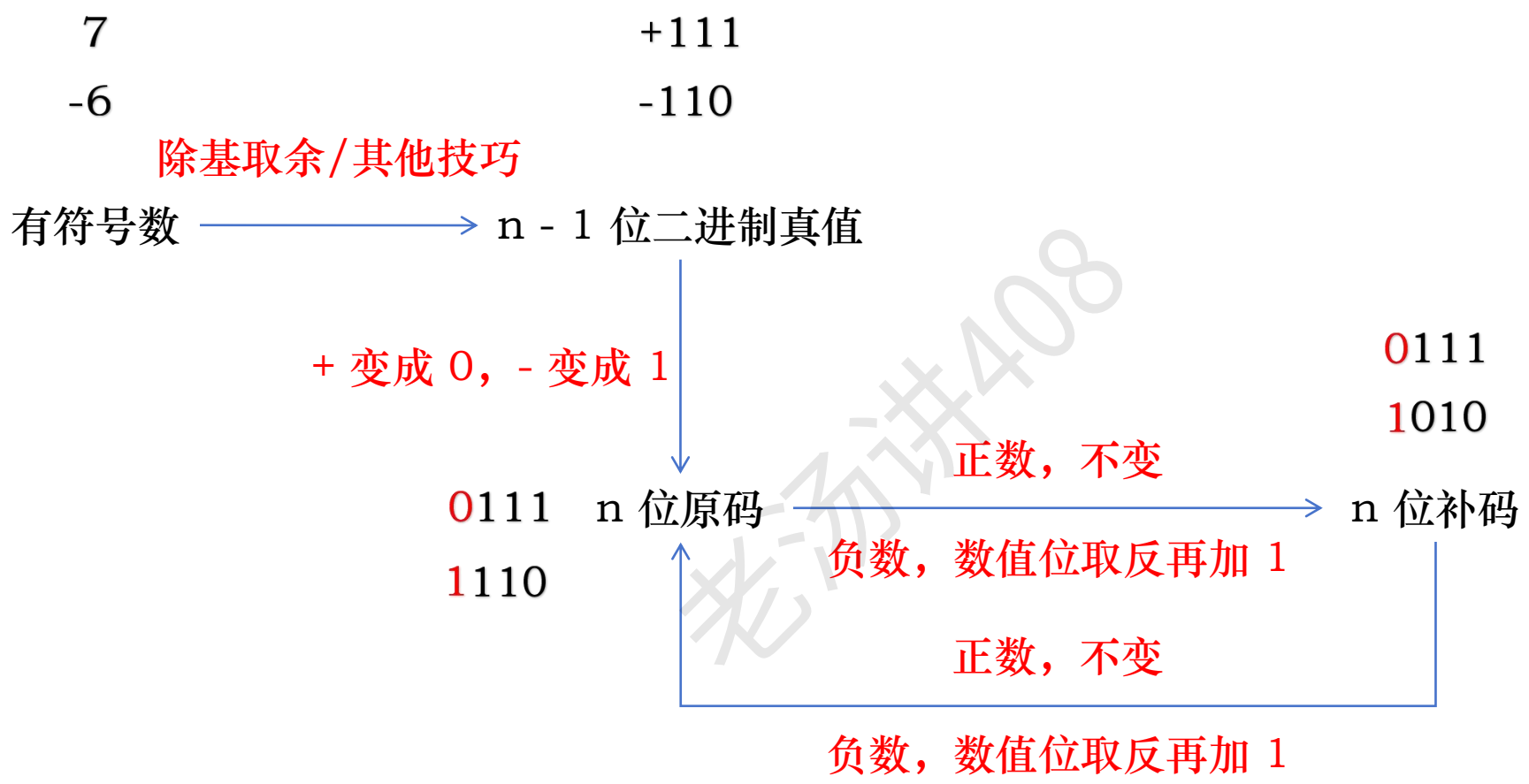
	补码
7	0111
6	0110
5	0101
4	0100
3	0011
2	0010
1	0001
0	0000
-1	1111
-2	1110
-3	1101
-4	1100
-5	1011
-6	1010
-7	1001
-8	1000

有符号整数的编码：补码



	补码
7	0111
6	0110
5	0101
4	0100
3	0011
2	0010
1	0001
0	0000
-1	1111
-2	1110
-3	1101
-4	1100
-5	1011
-6	1010
-7	1001
-8	1000

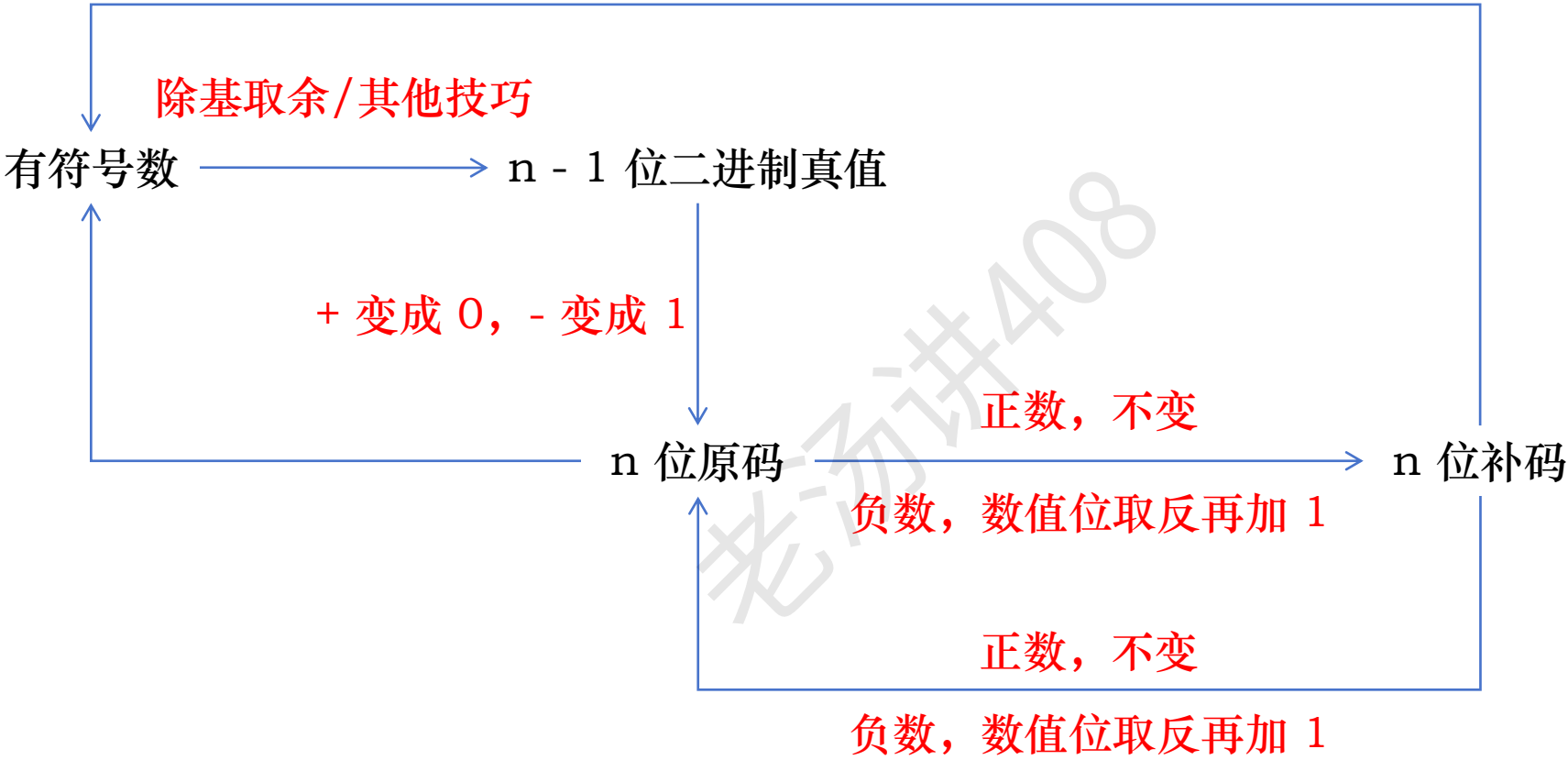
有符号整数的编码：补码



	补码
7	0111
6	0110
5	0101
4	0100
3	0011
2	0010
1	0001
0	0000
-1	1111
-2	1110
-3	1101
-4	1100
-5	1011
-6	1010
-7	1001
-8	1000

有符号整数的编码：补码

补码对应整数： $-x_{n-1} * 2^{n-1} + x_{n-2} * 2^{n-2} + \dots + x_2 * 2^2 + x_1 * 2^1 + x_0 * 2^0 = -x_{n-1} * 2^{n-1} + \sum_{i=0}^{n-2} x_i * 2^i$



原码对应整数： $(-1)^{x_{n-1}} * (x_{n-2} * 2^{n-2} + \dots + x_2 * 2^2 + x_1 * 2^1 + x_0 * 2^0) = (-1)^{x_{n-1}} * (\sum_{i=0}^{n-2} x_i * 2^i)$

有符号整数的编码：补码

n 位二进制： $x_{n-1}x_{n-2}...x_2x_1x_0$



有符号整数： $-x_{n-1} * 2^{n-1} + x_{n-2} * 2^{n-2} + ... + x_2 * 2^2 + x_1 * 2^1 + x_0 * 2^0 = -x_{n-1} * 2^{n-1} + \sum_{i=0}^{n-2} x_i * 2^i$

1011对应整数： $-1 * 2^3 + 0 * 2^2 + 1 * 2^1 + 1 * 2^0 = -5$

0011对应整数： $0 * 2^3 + 0 * 2^2 + 1 * 2^1 + 1 * 2^0 = 3$

		补码
7	←	0111
6	←	0110
5	←	0101
4	←	0100
3	←	0011
2	←	0010
1	←	0001
0	←	0000
-1	←	1111
-2	←	1110
-3	←	1101
-4	←	1100
-5	←	1011
-6	←	1010
-7	←	1001
-8	←	1000

有符号整数的编码：补码

无符号整数： $1 * 2^3 + 0 * 2^2 + 1 * 2^1 + 1 * 2^0 = 11$

无符号整数： $x_{n-1} * 2^{n-1} + x_{n-2} * 2^{n-2} + \dots + x_2 * 2^2 + x_1 * 2^1 + x_0 * 2^0$ $= \sum_{i=0}^{n-1} x_i * 2^i$



无符号编码

n 位二进制： $x_{n-1}x_{n-2}\dots x_2x_1x_0$ 1011



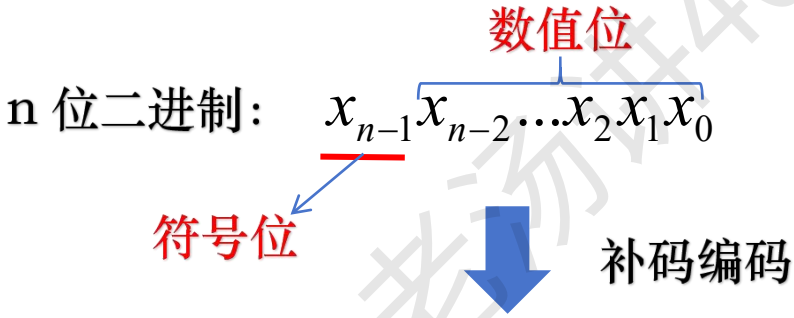
补码编码

最高位的权重变成负数

有符号整数： $-x_{n-1} * 2^{n-1} + x_{n-2} * 2^{n-2} + \dots + x_2 * 2^2 + x_1 * 2^1 + x_0 * 2^0$ $= -x_{n-1} * 2^{n-1} + \sum_{i=0}^{n-2} x_i * 2^i$

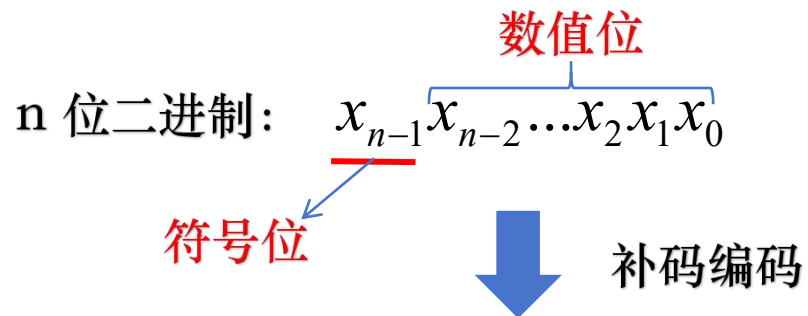
有符号整数： $-1 * 2^3 + 0 * 2^2 + 1 * 2^1 + 1 * 2^0 = -5$

有符号整数的编码：补码



有符号整数: $\underbrace{-x_{n-1} * 2^{n-1}}_{-2^{n-1}} + \underbrace{x_{n-2} * 2^{n-2} + \dots + x_2 * 2^2 + x_1 * 2^1 + x_0 * 2^0}_{2^{n-1} - 1} = -x_{n-1} * 2^{n-1} + \sum_{i=0}^{n-2} x_i * 2^i$

有符号整数的编码：补码的表示范围



$$\text{有符号整数: } \underline{-x_{n-1} * 2^{n-1}} + x_{n-2} * 2^{n-2} + \dots + x_2 * 2^2 + x_1 * 2^1 + x_0 * 2^0 = -x_{n-1} * 2^{n-1} + \sum_{i=0}^{n-2} x_i * 2^i$$

最小值：最高位为 1，其他位是 0 $= -2^{n-1}$

最大值：最高位为 0，其他位是 1 $= 2^{n-1} - 1$

n 位补码的表示范围： $-2^{n-1} \sim 2^{n-1} - 1$

机器字长 = n n 位原码表示有符号整数的范围： $-(2^{n-1} - 1) \sim 2^{n-1} - 1$

有符号整数的编码：补码的表示范围

+0 原码：000...000

-0 原码：100...000

$$(-1)^{x_{n-1}} * (x_{n-2} * 2^{n-2} + \dots + x_2 * 2^2 + x_1 * 2^1 + x_0 * 2^0) = (-1)^{x_{n-1}} * \left(\sum_{i=0}^{n-2} x_i * 2^i \right)$$



原码编码

n 位二进制： $x_{n-1}x_{n-2}\dots x_2x_1x_0$



补码编码

$$-x_{n-1} * 2^{n-1} + x_{n-2} * 2^{n-2} + \dots + x_2 * 2^2 + x_1 * 2^1 + x_0 * 2^0 = -x_{n-1} * 2^{n-1} + \sum_{i=0}^{n-2} x_i * 2^i$$

+0

补码：000...000

在补码中 100...000 表示最小值，也就是 -2^{n-1}

-0

n 位补码的表示范围： $-2^{n-1} \sim 2^{n-1} - 1$

n 位原码表示范围： $-(2^{n-1} - 1) \sim 2^{n-1} - 1$

有符号整数的编码：补码的表示范围

$$(-1)^{x_{n-1}} * (x_{n-2} * 2^{n-2} + \dots + x_2 * 2^2 + x_1 * 2^1 + x_0 * 2^0) = (-1)^{x_{n-1}} * \left(\sum_{i=0}^{n-2} x_i * 2^i \right)$$



原码编码

n 位二进制: $x_{n-1}x_{n-2}\dots x_2x_1x_0$



补码编码

$$-x_{n-1} * 2^{n-1} + x_{n-2} * 2^{n-2} + \dots + x_2 * 2^2 + x_1 * 2^1 + x_0 * 2^0 = -x_{n-1} * 2^{n-1} + \sum_{i=0}^{n-2} x_i * 2^i$$

当原码和补码的最高位都是 0 的时候，它们表示的是同一个正整数

补码和原码之间的转换

机器字长 = 8 位

-1 的补码

原码: 10000001

数值位取反: 11111110

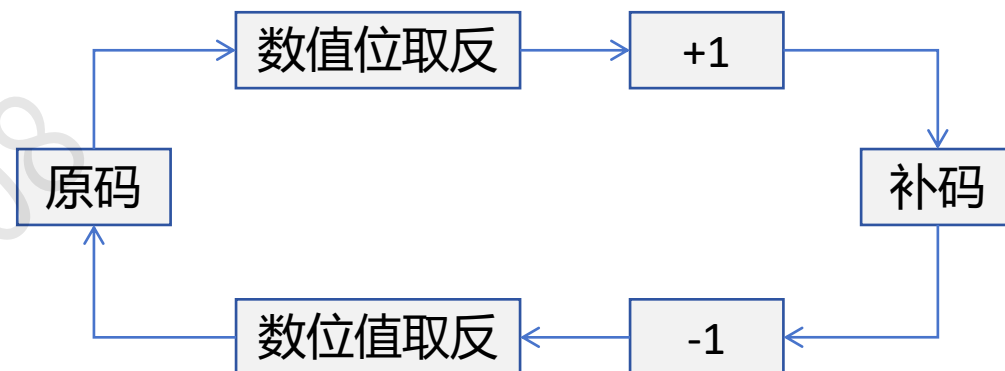
+1: 11111111

-127 的补码

原码: 11111111

数值位取反: 10000000

+1: 10000001



补码: 10001010

-1: 10001001

数值位取反: 11110110

原码

补码: 10001010

数值位取反: 11110101

+1: 11110110

原码

补码和原码之间的转换

机器字长 = 16 位

补码: 01110001 10101010 $\longrightarrow$ 原码: 01110001 10101010

补码: 11110001 10100000	原码	原码: 11110001 10100000	补码
数值位取反: 10001110 01011111		数值位取反: 10001110 01011111	
+ : 10001110 01100000 $\longleftarrow$		+1: 10001110 01100000 $\longleftarrow$	

补码和原码之间的转换

机器字长 = 16 位

补码: 01110001 10101010 $\longrightarrow$ 原码: 01110001 10101010

补码: 11110001 10 <u>100000</u>	原码	原码: 11110001 10 <u>100000</u>	补码
数值位取反: 10001110 01011111		数值位取反: 10001110 01011111	
+1: 10001110 01 <u>100000</u>		+1: 10001110 01 <u>100000</u>	

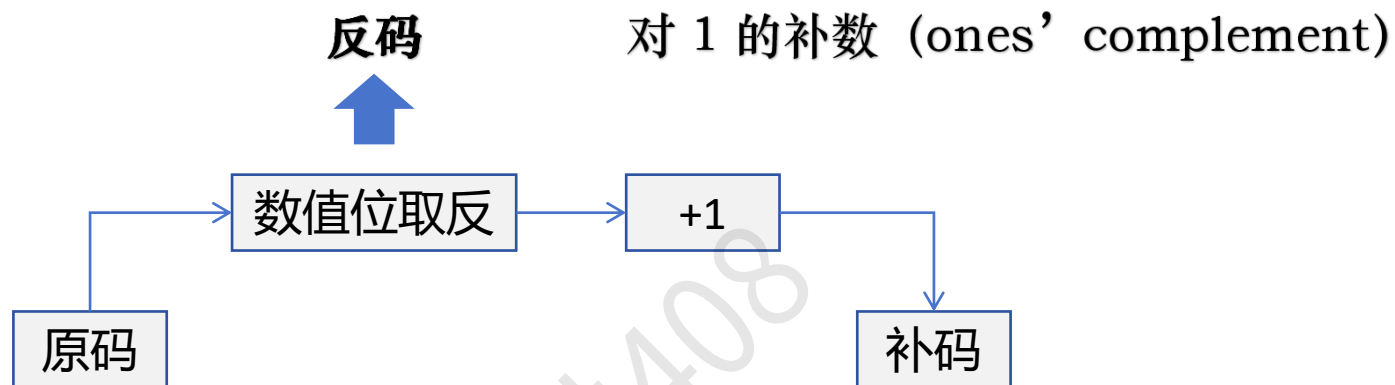
原码: 11110001 10000000

补码: 10001110 10000000

补码: 11110001 10000000

原码: 10001110 10000000

有符号整数编码：反码和移码



正数：原码 = 反码 = 补码

负数：原码、反码和补码的表示都不同，但是它们的最高位都是符号位，且都是 1

整数	真值	原码	反码	补码
-8	-1000			1000
-7	-111	1111	1000	1001
-6	-110	1110	1001	1010
-5	-101	1101	1010	1011
-4	-100	1100	1011	1100
-3	-011	1011	1100	1101
-2	-010	1010	1101	1110
-1	-001	1001	1110	1111
-0	-000	1000	1111	0000
+0	+000	0000	0111	0000
1	+001	0001	0001	0001
2	+010	0010	0010	0010
3	+011	0011	0011	0011
4	+100	0100	0100	0100
5	+101	0101	0101	0101
6	+110	0110	0110	0110
7	+111	0111	0111	0111

机器字长 = 4 位

正数的话，补码等于反码等于原码

负数的话，反码等于原码的数值位取反

负数的话，补码等于反码加 1

n 位原码表示范围： $-(2^{n-1} - 1) \sim 2^{n-1} - 1$

n 位反码表示范围： $-(2^{n-1} - 1) \sim 2^{n-1} - 1$

n 位补码的表示范围： $-2^{n-1} \sim 2^{n-1} - 1$

机器字长 = n 位 n 位二进制: $x_{n-1}x_{n-2}\dots x_2x_1x_0$

无符号编码: $x_{n-1} * 2^{n-1} + x_{n-2} * 2^{n-2} + \dots + x_2 * 2^2 + x_1 * 2^1 + x_0 * 2^0 = \sum_{i=0}^{n-1} x_i * 2^i$

原码: $(-1)^{x_{n-1}} * (x_{n-2} * 2^{n-2} + \dots + x_2 * 2^2 + x_1 * 2^1 + x_0 * 2^0) = (-1)^{x_{n-1}} * (\sum_{i=0}^{n-2} x_i * 2^i)$

补码: $-x_{n-1} * 2^{n-1} + x_{n-2} * 2^{n-2} + \dots + x_2 * 2^2 + x_1 * 2^1 + x_0 * 2^0 = -x_{n-1} * 2^{n-1} + \sum_{i=0}^{n-2} x_i * 2^i$

反码: $-x_{n-1} * (2^{n-1} - 1) + x_{n-2} * 2^{n-2} + \dots + x_2 * 2^2 + x_1 * 2^1 + x_0 * 2^0 = -x_{n-1} * 2^{n-1} + \sum_{i=0}^{n-2} x_i * 2^i$

整数	真值	原码	反码	补码
-8	-1000			1000
-7	-111	1111	1000	1001
-6	-110	1110	1001	1010
-5	-101	1101	1010	1011
-4	-100	1100	1011	1100
-3	-011	1011	1100	1101
-2	-010	1010	1101	1110
-1	-001	1001	1110	1111
-0	-000	1000	1111	0000
+0	+000	0000	0111	0000
1	+001	0001	0001	0001
2	+010	0010	0010	0010
3	+011	0011	0011	0011
4	+100	0100	0100	0100
5	+101	0101	0101	0101
6	+110	0110	0110	0110
7	+111	0111	0111	0111

机器字长 = 4 位

正数的话，补码等于反码等于原码

负数的话，反码等于原码的数值位取反

负数的话，补码等于反码加 1

n 位原码表示范围： $-(2^{n-1} - 1) \sim 2^{n-1} - 1$

n 位原码表示范围： $-(2^{n-1} - 1) \sim 2^{n-1} - 1$

n 位补码的表示范围： $-2^{n-1} \sim 2^{n-1} - 1$

机器字长 = n 位 n 位二进制: $x_{n-1}x_{n-2}\dots x_2x_1x_0$

无符号编码: $x_{n-1} * 2^{n-1} + x_{n-2} * 2^{n-2} + \dots + x_2 * 2^2 + x_1 * 2^1 + x_0 * 2^0 = \sum_{i=0}^{n-1} x_i * 2^i$

原码: $(-1)^{x_{n-1}} * (x_{n-2} * 2^{n-2} + \dots + x_2 * 2^2 + x_1 * 2^1 + x_0 * 2^0) = (-1)^{x_{n-1}} * (\sum_{i=0}^{n-2} x_i * 2^i)$

补码: $-x_{n-1} * 2^{n-1} + x_{n-2} * 2^{n-2} + \dots + x_2 * 2^2 + x_1 * 2^1 + x_0 * 2^0 = -x_{n-1} * 2^{n-1} + \sum_{i=0}^{n-2} x_i * 2^i$

反码: $-x_{n-1} * (2^{n-1} - 1) + x_{n-2} * 2^{n-2} + \dots + x_2 * 2^2 + x_1 * 2^1 + x_0 * 2^0 = -x_{n-1} * 2^{n-1} + \sum_{i=0}^{n-2} x_i * 2^i$

整数	真值	原码	反码	补码
-8	-1000			1000
-7	-111	1111	1000	1001
-6	-110	1110	1001	1010
-5	-101	1101	1010	1011
-4	-100	1100	1011	1100
-3	-011	1011	1100	1101
-2	-010	1010	1101	1110
-1	-001	1001	1110	1111
-0	-000	1000	1111	0000
+0	+000	0000	0111	0000
1	+001	0001	0001	0001
2	+010	0010	0010	0010
3	+011	0011	0011	0011
4	+100	0100	0100	0100
5	+101	0101	0101	0101
6	+110	0110	0110	0110
7	+111	0111	0111	0111

机器字长 = 4 位

正数的话，补码等于反码等于原码

负数的话，反码等于原码的数值位取反

负数的话，补码等于反码加 1

n 位原码表示范围： $-(2^{n-1} - 1) \sim 2^{n-1} - 1$

n 位原码表示范围： $-(2^{n-1} - 1) \sim 2^{n-1} - 1$

n 位补码的表示范围： $-2^{n-1} \sim 2^{n-1} - 1$

机器字长 = n 位 n 位二进制: $x_{n-1}x_{n-2}\dots x_2x_1x_0$

无符号编码: $x_{n-1} * 2^{n-1} + x_{n-2} * 2^{n-2} + \dots + x_2 * 2^2 + x_1 * 2^1 + x_0 * 2^0 = \sum_{i=0}^{n-1} x_i * 2^i$

原码: $(-1)^{x_{n-1}} * (x_{n-2} * 2^{n-2} + \dots + x_2 * 2^2 + x_1 * 2^1 + x_0 * 2^0) = (-1)^{x_{n-1}} * (\sum_{i=0}^{n-2} x_i * 2^i)$

补码: $-x_{n-1} * 2^{n-1} + x_{n-2} * 2^{n-2} + \dots + x_2 * 2^2 + x_1 * 2^1 + x_0 * 2^0 = -x_{n-1} * 2^{n-1} + \sum_{i=0}^{n-2} x_i * 2^i$

反码: $-x_{n-1} * (2^{n-1} - 1) + x_{n-2} * 2^{n-2} + \dots + x_2 * 2^2 + x_1 * 2^1 + x_0 * 2^0 = -x_{n-1} * 2^{n-1} + \sum_{i=0}^{n-2} x_i * 2^i$

机器字长 = 8 位

8 位二进制	无符号数对应真值	原码对应的真值	反码对应的真值	补码对应的真值
0000 0000	0	+0	+0	± 0
0000 0001	1	+1	+1	+1
0000 0010	2	+2	+2	+2
...	...	...	...	...
0111 1110	126	+126	+126	+126
0111 1111	127	+127	+127	+127
1000 0000	128	-0	-127	-128
1000 0001	129	-1	-126	-127
1000 0010	130	-2	-125	-126
...	...	...	...	...
1111 1101	253	-125	-2	-3
1111 1110	254	-126	-1	-2
1111 1111	255	-127	-0	-1

有符号整数编码：移码

	真值	补码
21	+10101	010101
-21	-10101	101011
31	+11111	011111
-31	-11111	100001

二进制 1 比二进制 0 大

010101 < 011111 < 100001 < 101011
21 31 -31 -21

有符号整数编码：移码

	真值	补码	移码 = 真值 + 2^5	$\begin{array}{r} 100000 \\ - 10101 \\ \hline 001011 \end{array}$
21	+10101	010101	$+10101 + 100000 = 110101$	
-21	-10101	101011	$-10101 + 100000 = 001011$	
31	+11111	011111	$+11111 + 100000 = 111111$	
-31	-11111	100001	$-11111 + 100000 = 000001$	

二进制 1 比二进制 0 大

$$\begin{array}{ccccccc} 000001 & < & 001011 & < & 110101 & < & 111111 \\ -31 & & -21 & & 21 & & 31 \end{array}$$

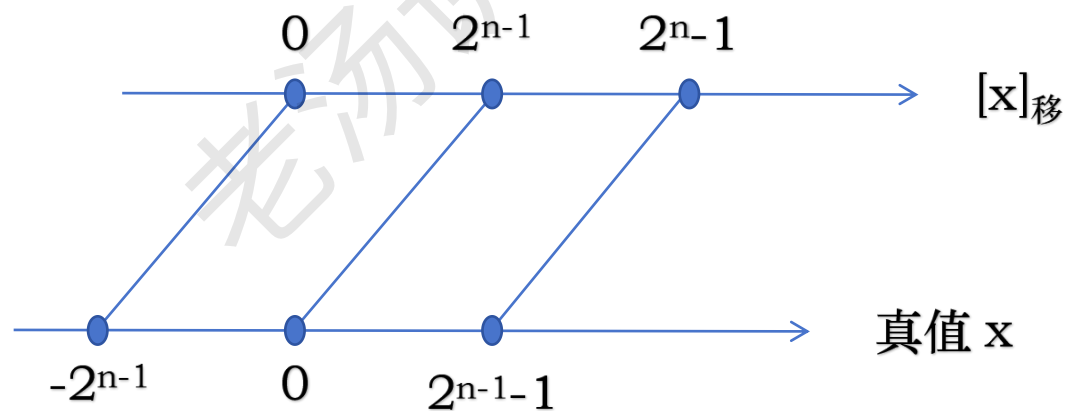
有符号整数编码：移码

$$[x]_{\text{移}} = 2^{n-1} + x \quad x \text{ 是真值, } n \text{ 是 } x \text{ 原码的位数, 也是移码的位数}$$

	真值	补码	移码 = 真值 + 2^5
21	+10101	010101	+10101 + 100000 = 110101
-21	-10101	101011	-10101 + 100000 = 001011
31	+11111	011111	+11111 + 100000 = 111111
-31	-11111	100001	-11111 + 100000 = 000001

有符号整数编码：移码

$[x]_{\text{移}} = 2^{n-1} + x$ x 是真值， n 是 x 原码的位数，也是移码的位数



移码可以表达有符号整数的范围是： -2^{n-1} 到 $2^{n-1} - 1$

有符号整数编码：移码

编码方式	最小值	最大值
无符号编码	0	$2^n - 1$
原码	$-(2^{n-1} - 1)$	$2^{n-1} - 1$
补码	-2^{n-1}	$2^{n-1} - 1$
移码	-2^{n-1}	$2^{n-1} - 1$

$$[+0]_{\text{移}} = 2^{n-1} + 0 = 2^{n-1}$$

$$[-0]_{\text{移}} = 2^{n-1} - 0 = 2^{n-1}$$

有符号整数编码：移码

对于相同的真值，补码和移码的符号位相反，但是数值位是一样的

	真值	补码	移码 = 真值 + 2^5
21	+10101	010101	+10101 + 100000 = 110101
-21	-10101	101011	-10101 + 100000 = 001011
31	+11111	011111	+11111 + 100000 = 111111
-31	-11111	100001	-11111 + 100000 = 000001

有符号整数编码：移码

机器字长 = 8 位

真值 x	$[x]_{\text{补}}$	$[x]_{\text{移}}$	$[x]_{\text{移}}$ 对应的 十进制整数
-100000	100000	000000	0
-11111	100001	000001	1
-11110	100010	000010	2
⋮	⋮	⋮	⋮
-00001	111111	011111	31
±00000	000000	100000	32
+00001	000001	100001	33
+00010	000010	100010	34
⋮	⋮	⋮	⋮
+11110	011110	111110	62
+11111	011111	111111	63

C 语言中的有符号数类型

C 语言中的无符号整数

C 声明类型	32 位程序	64 位程序
unsigned char	1 字节	1 字节
unsigned short	2 字节	2 字节
unsigned int	4 字节	4 字节
unsigned long	4 字节	8 字节



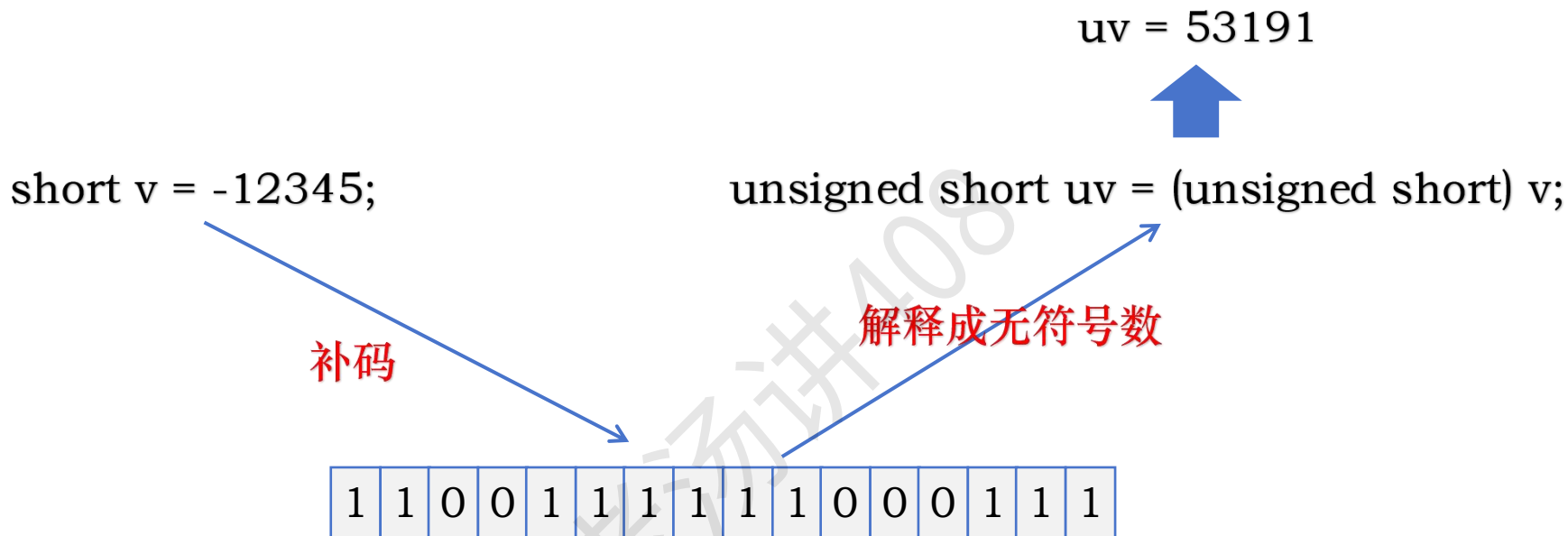
C 语言中的有符号整数

采用补码表示

C 声明类型	32 位程序	64 位程序
char	1 字节	1 字节
short	2 字节	2 字节
int	4 字节	4 字节
long	4 字节	8 字节

有符号数和无符号数之间的转换

保持机器码位值不变，只是改变了解释机器码位值的方式而已



无符号整数: $x_{n-1} * 2^{n-1} + x_{n-2} * 2^{n-2} + \dots + x_2 * 2^2 + x_1 * 2^1 + x_0 * 2^0 = \sum_{i=0}^{n-1} x_i * 2^i$

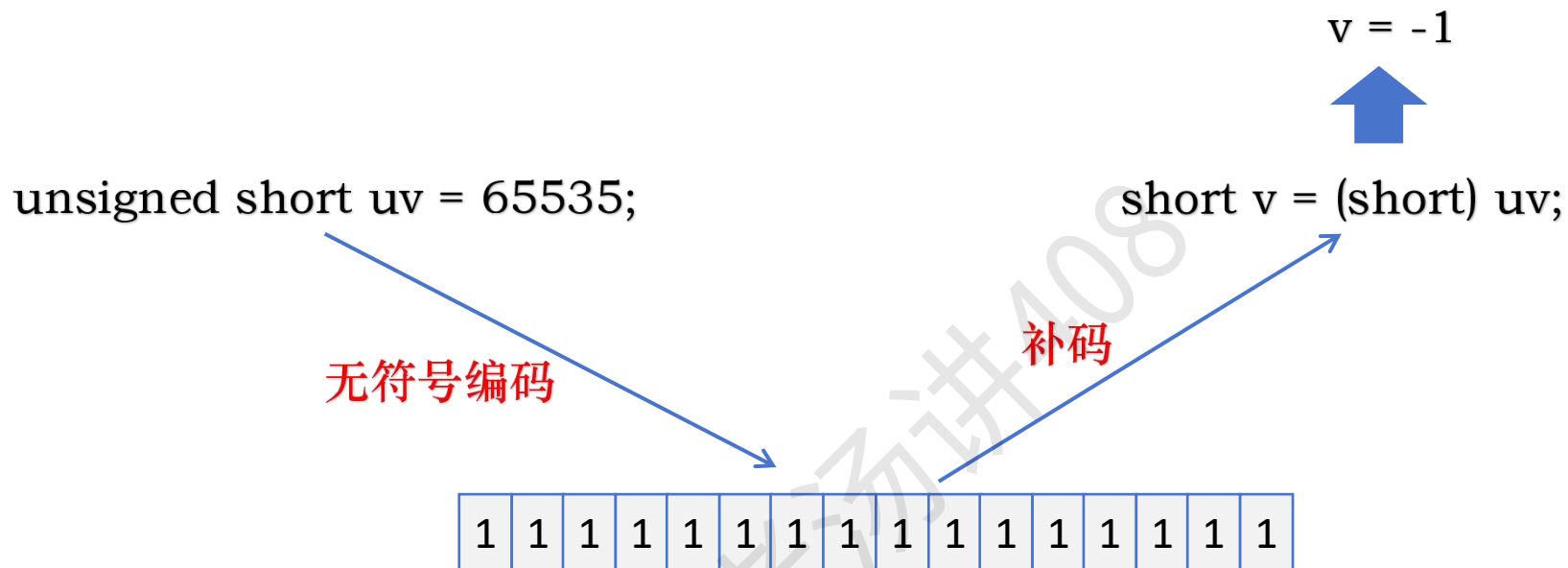
补码: $-x_{n-1} * 2^{n-1} + x_{n-2} * 2^{n-2} + \dots + x_2 * 2^2 + x_1 * 2^1 + x_0 * 2^0 = -x_{n-1} * 2^{n-1} + \sum_{i=0}^{n-2} x_i * 2^i$

$v < 0$ $v + 2^n$ n 是补码的位数 $-12345 + 2^{16} = -12345 + 65536 = 53191$

$v \geq 0$ 无符号数就等于补码

有符号数和无符号数之间的转换

保持机器码位值不变，只是改变了解释机器码位值的方式而已



$$\text{补码: } -x_{n-1} * 2^{n-1} + x_{n-2} * 2^{n-2} + \dots + x_2 * 2^2 + x_1 * 2^1 + x_0 * 2^0 = -x_{n-1} * 2^{n-1} + \sum_{i=0}^{n-2} x_i * 2^i$$

$$\text{无符号整数: } x_{n-1} * 2^{n-1} + x_{n-2} * 2^{n-2} + \dots + x_2 * 2^2 + x_1 * 2^1 + x_0 * 2^0 = \sum_{i=0}^{n-1} x_i * 2^i$$

n 位补码表示范围: $-2^{n-1} \sim 2^{n-1}-1$

16 位补码表示范围: $-32768 \sim 32767$

$$uv > 2^{n-1}-1$$

$$uv - 2^n$$

n 是无符号编码的位数

$$65535 - 2^{16} = 65535 - 65536 = -1$$

$$uv \leq 2^{n-1}-1$$

$$uv$$

有符号数和无符号数之间隐式转换

```
short v = -12345;
```

```
unsigned short uv = (unsigned short) v;
```

```
unsigned short uv = v;
```

```
unsigned short uv = 65535u;
```

```
short v = (short) uv;
```

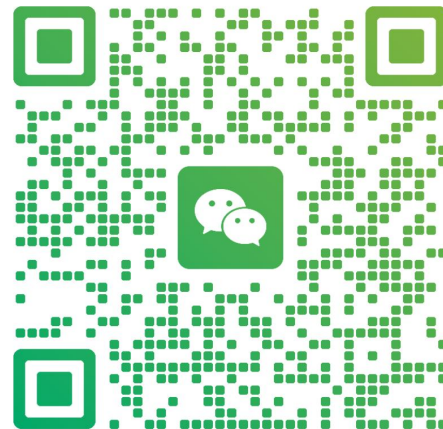
```
short v = uv;
```

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

有符号数和无符号数之间隐式转换

在 C 语言中，在执行一个运算时，如果它的一个运算数是有符号的，而另一个是无符号的，那么 C 语言会隐式的将有符号数强制类型转换为无符号数

$$-1 + 2^{32} = 4\ 294\ 967\ 295$$

```
int flag = -1 < 0u;  4 294 967 295u < 0u  
printf("flag = %d", flag);
```

```
flag = 0
```

零扩展

符号扩展

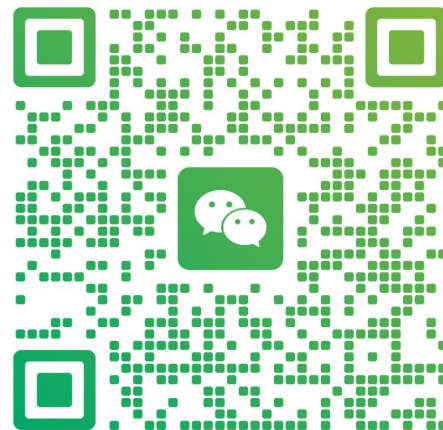
位截断

加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



老汤

浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

位扩展一个无符号数：零扩展

ui = 53191u

unsigned short us = 53191u; → unsigned int ui = us;

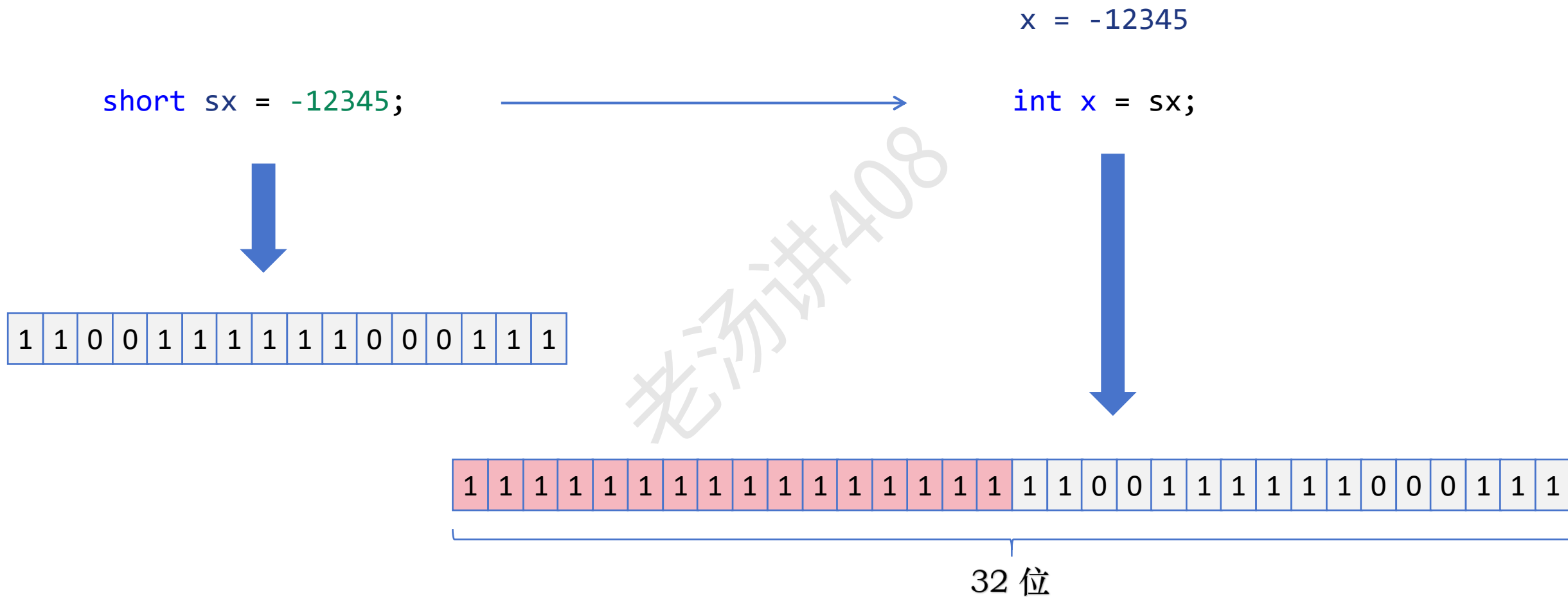
1 1 0 0 1 1 1 1 1 1 0 0 0 1 1 1

0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 1 1 1 1 1 1 0 0 0 1 1 1

32 位

如果将一个无符号数转换成一个更大的无符号类型的话，就需要执行零扩展

位扩展一个有符号数：符号扩展



如果将一个有符号数转换成一个更大的有符号类型的话，就需要执行符号扩展

位扩展一个有符号数：符号扩展

x = 12345

short sx = 12345;

int x = sx;

0 0 1 1 0 0 0 0 0 0 1 1 1 0 0 1

0 1 1 0 0 0 0 0 0 0 1 1 1 0 0 1

32 位

无符号数和有符号数之间的转换顺序



(unsigned int) (unsigned short) ux

(unsigned) (int) ux

short sx = -12345;

unsigned int ux = sx;

补码

1	1	0	0	1	1	1	1	1	1	0	0	0	1	1	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

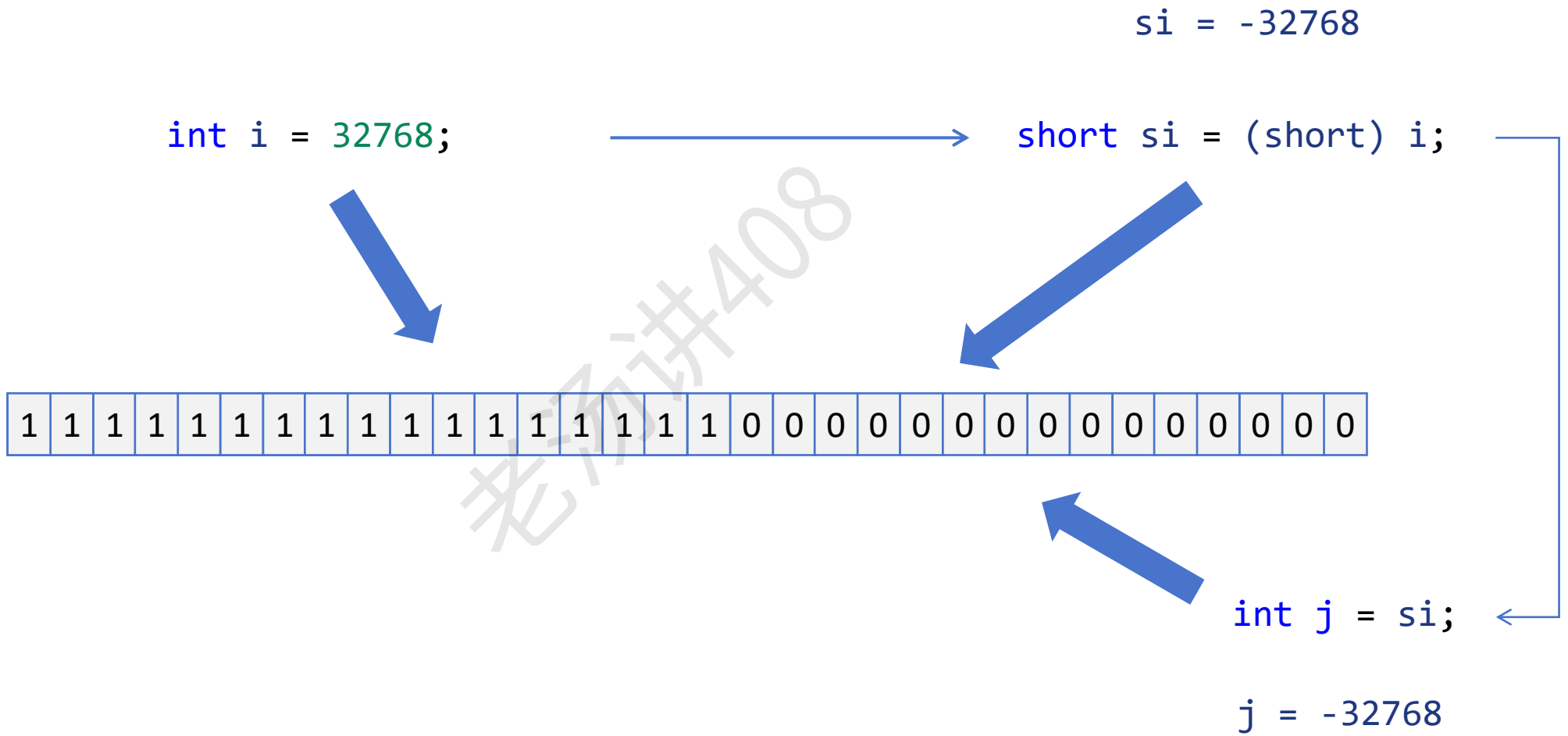
符号扩展

1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	0	1	1	1	1	1	1	0	0	0	1	1	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

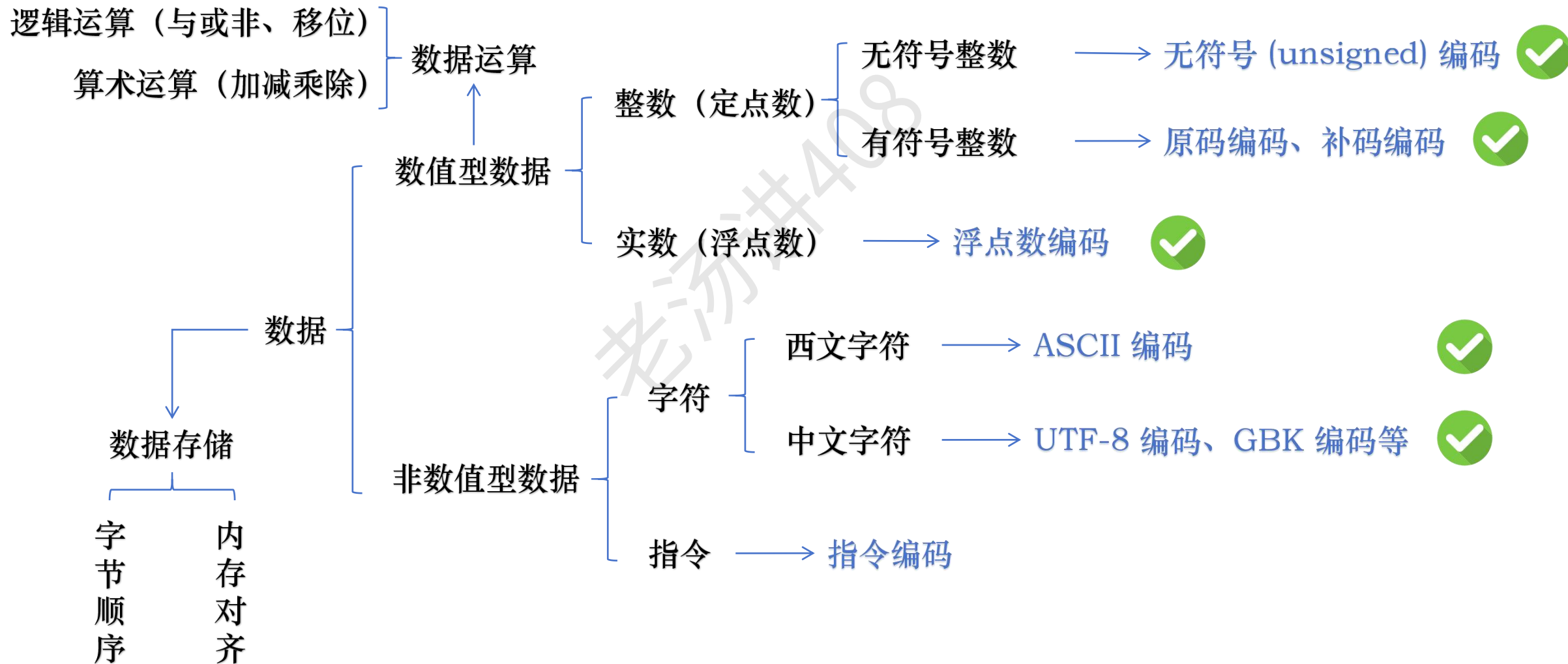
解释成无符号数

ux = 4294954951

位截断



浮点数编码



定点数和浮点数

约定小数点的位置来实现

**加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】**

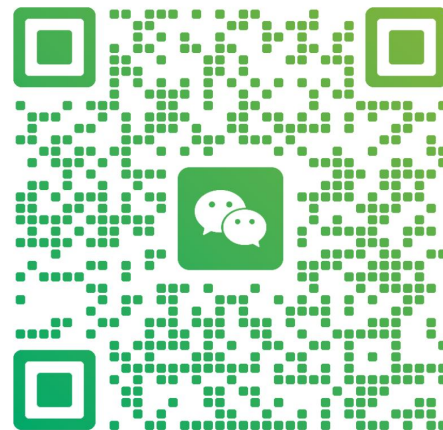
小数点位置约定在固定位置的数称为定点数

小数点位置约定为可浮动的数称为浮点数



老汤

浙江 杭州

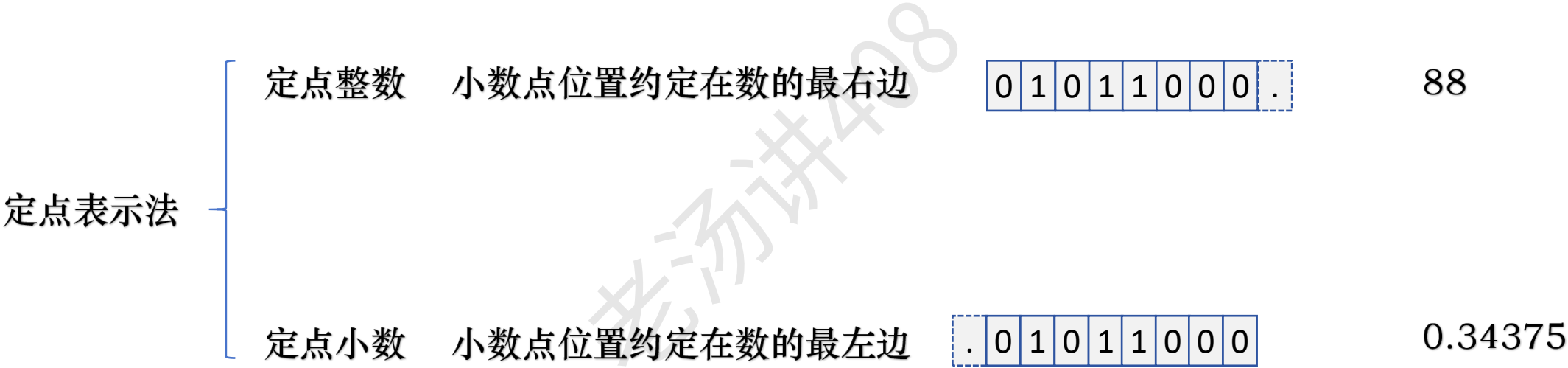


扫一扫上面的二维码图案，加我为朋友。

小数点位置约定在固定位置的数称为定点数

定点表示

机器字长 = 8 位



定点小数：二进制转十进制

0	1	0	1	1	0	0	0	.
---	---	---	---	---	---	---	---	---



$$0 * 2^7 + 1 * 2^6 + 0 * 2^5 + 1 * 2^4 + 1 * 2^3 + 0 * 2^2 + 0 * 2^1 + 0 * 2^0 = 88$$

.	0	1	0	1	1	0	0	0
---	---	---	---	---	---	---	---	---



$$0 * 2^{-1} + 1 * 2^{-2} + 0 * 2^{-3} + 1 * 2^{-4} + 1 * 2^{-5} + 0 * 2^{-6} + 0 * 2^{-7} + 0 * 2^{-8} = 0.34375$$

定点小数：二进制转十进制

10101.011B



$$1 * 2^4 + 0 * 2^3 + 1 * 2^2 + 0 * 2^1 + 1 * 2^0 + 0 * 2^{-1} + 1 * 2^{-2} + 1 * 2^{-3} = 21.375$$



$$1355.34 = 0.135534 \times 10^4$$

$$10101.011B = 0.10101011 \times 2^5$$

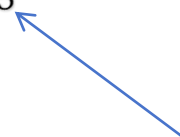


浮点数



定点小数

定点整数



定点小数：二进制转十进制

$$x_{n-1} \dots x_2 x_1 x_0 . x_{-1} x_{-2} \dots x_{-m} B$$



$$x_{n-1} * 2^{n-1} + \dots + x_2 * 2^2 + x_1 * 2^1 + x_0 * 2^0 + x_{-1} * 2^{-1} + x_{-2} * 2^{-2} + \dots + x_{-m} * 2^{-m}$$



小数：十进制转二进制

$$135.6875 = 1000\ 0111.1011\text{B}$$

除 2 取余

余数	低位
1	↑
1	
1	
0	
0	
0	
0	
1	高位

1000 0111B

乘 2 取整

$$0.6875 \times 2 = 1.375$$

整数部分 = 1

$$0.375 \times 2 = 0.75$$

整数部分 = 0

$$0.75 \times 2 = 1.5$$

整数部分 = 1

$$0.5 \times 2 = 1.0$$

整数部分 = 1

高位

低位

0.1011B

小数：十进制转二进制

$$0.63 \approx 0.101000010B$$

高位

$$0.63 \times 2 = 1.26 \quad \text{整数部分} = 1$$

$$0.26 \times 2 = 0.52 \quad \text{整数部分} = 0$$

$$0.52 \times 2 = 1.04 \quad \text{整数部分} = 1$$

$$0.04 \times 2 = 0.08 \quad \text{整数部分} = 0$$

$$0.08 \times 2 = 0.16 \quad \text{整数部分} = 0$$

$$0.16 \times 2 = 0.32 \quad \text{整数部分} = 0$$

$$0.32 \times 2 = 0.64 \quad \text{整数部分} = 0$$

$$0.64 \times 2 = 1.28 \quad \text{整数部分} = 1$$

$$0.28 \times 2 = 0.56 \quad \text{整数部分} = 0$$

低位

.....

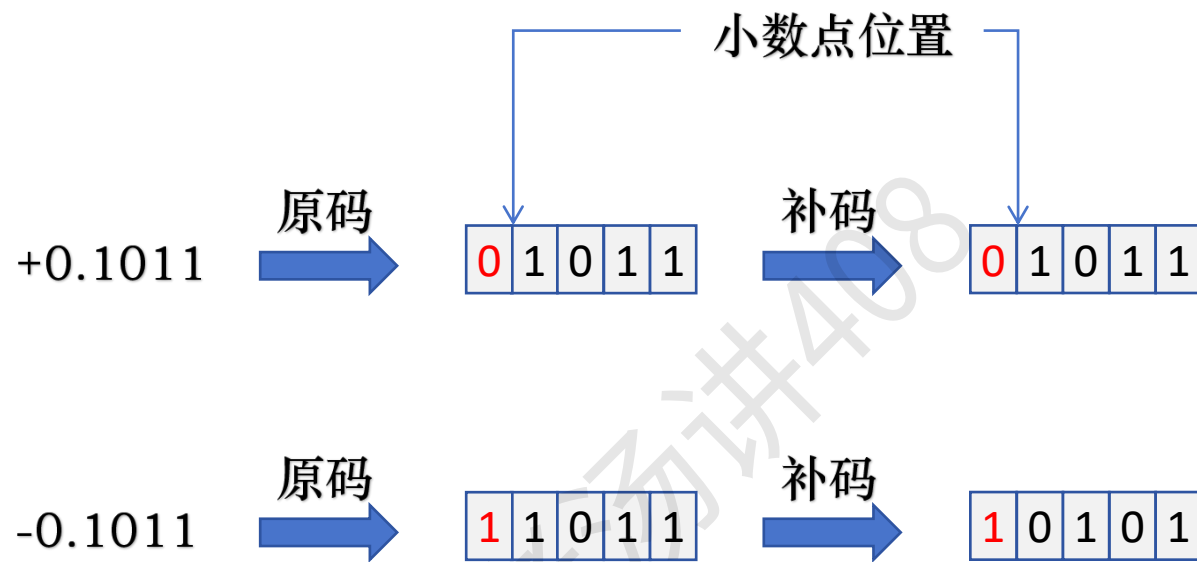
不是所有的十进制小数，
都可以使用二进制小数精确表示

小数：十进制转二进制

		高位
$0.4375 \times 2 = 0.875$	整数部分 = 0	
$0.875 \times 2 = 1.75$	整数部分 = 1	
$0.75 \times 2 = 1.5$	整数部分 = 1	
$0.5 \times 2 = 1.0$	整数部分 = 1	
		低位

不是所有的十进制小数，
都可以使用二进制小数精确表示

有符号定点小数的编码



浮点数编码

机器字长 = 32 位

135.6875



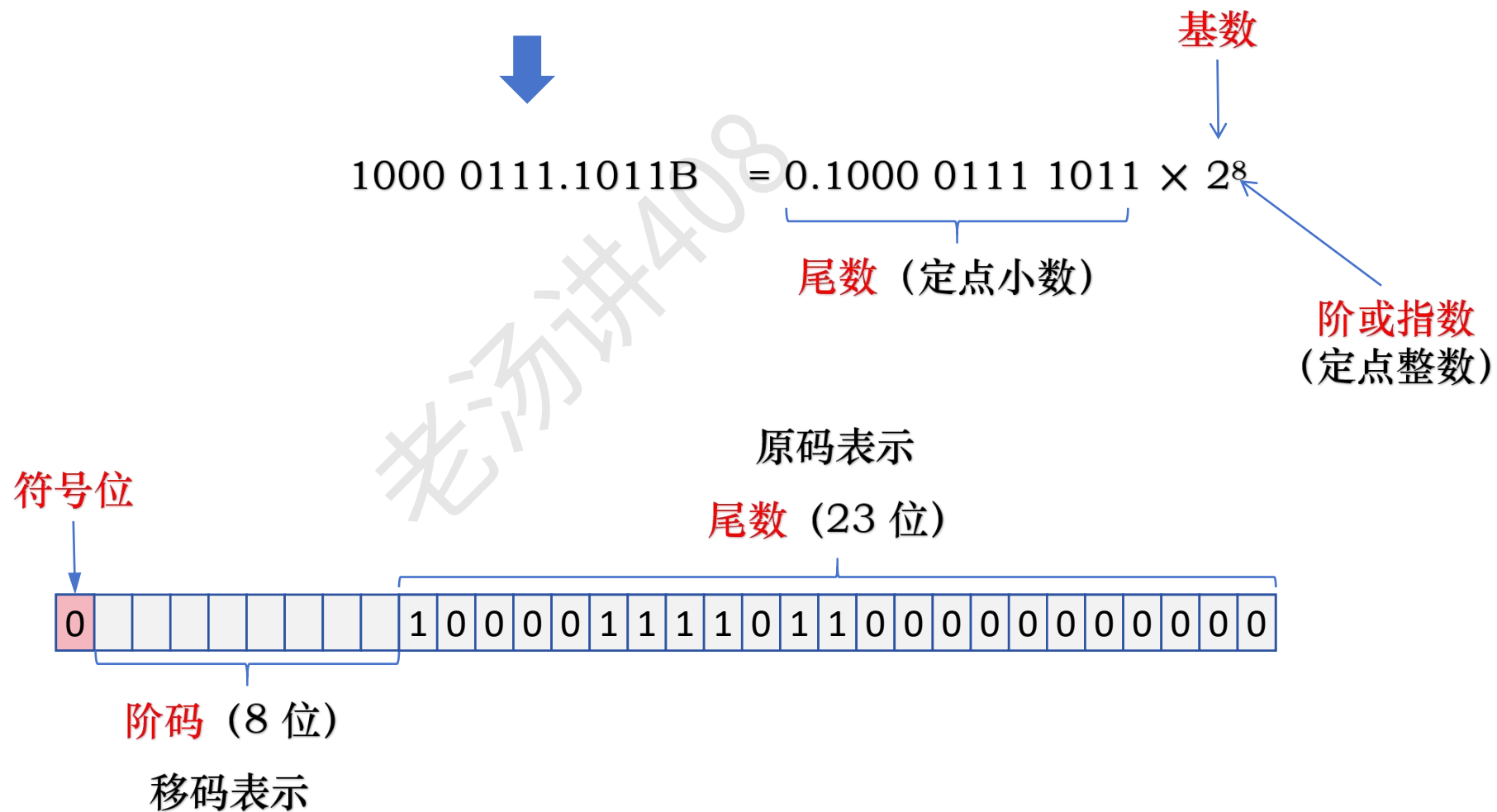
如何存储表示?



32 位

机器字长 = 32 位

135.6875



浮点数编码：阶码

机器字长 = 32 位

偏置值
↓
 $[x]_{\text{移}} = 2^{n-1} + x$

x 是真值，n 是 x 原码的位数，也是移码的位数

135.6875



1000 0111.1011B = 0.1000 0111 1011 × 2⁸

尾数 (定点小数)

基数

阶或指数
(定点整数)

原码表示

尾数 (23 位)

符号位



阶码 (8 位)

移码表示

偏置值: $2^{8-1} = 128$

$8 + 128 = 136 = 1000\ 1000\text{B}$

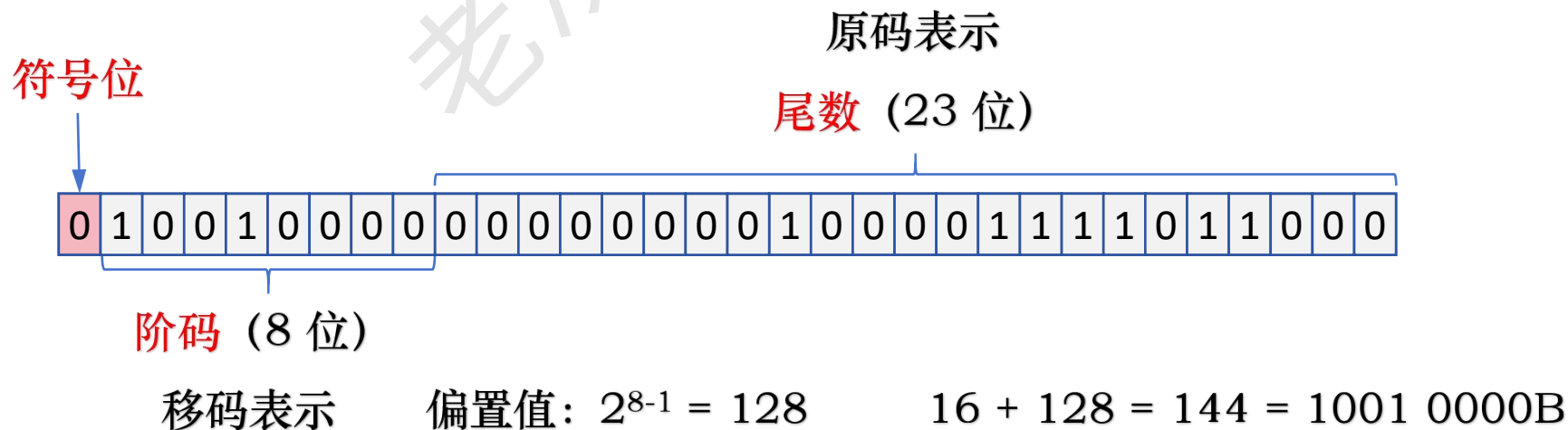
浮点数编码

机器字长 = 32 位

135.6875



$$\begin{aligned} 1000\ 0111.1011\text{B} &= 0.1000\ 0111\ 1011 \times 2^8 \\ &= 0.0000\ 0000\ 1000\ 0111\ 1011 \times 2^{16} \end{aligned}$$



浮点数编码

机器字长 = 32 位

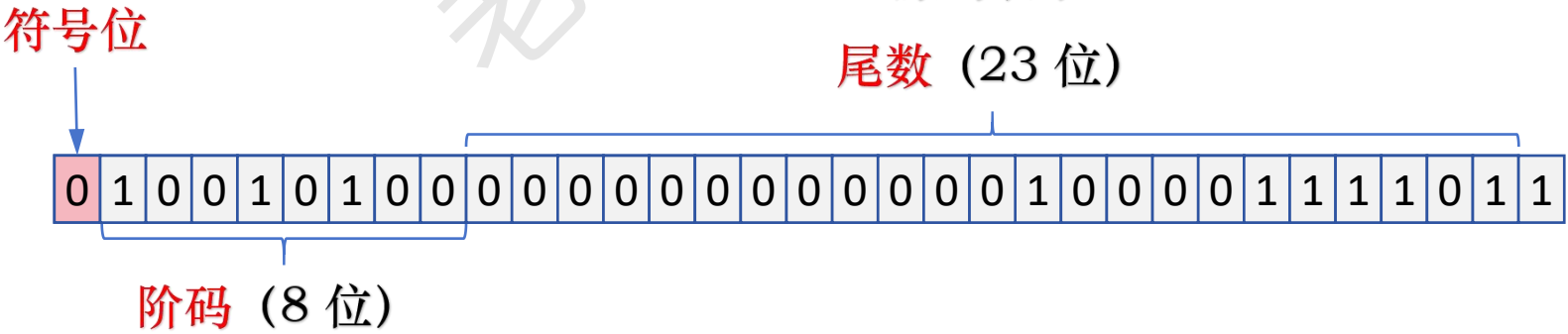
135.6875

135.625



1000 0111.1011B = 0.1000 0111 1011 $\times 2^8$
= 0.0000 0000 1000 0111 1011 $\times 2^{16}$
= 0.0000 0000 0000 1000 0111 1011 $\times 2^{20}$

原码表示



移码表示 偏置值: $2^{8-1} = 128$ $20 + 128 = 148 = 1001\ 0100\text{B}$

浮点数编码：规格化

机器字长 = 32 位

规格化浮点数的标志是尾数部分的最高位是 1

135.6875



1000 0111.1011B = 0.1000 0111 1011 $\times 2^8$

= 0.0000 0000 1000 0111 1011 $\times 2^{16}$

= 0.0000 0000 0000 1000 0111 1011 $\times 2^{20}$

原码表示

尾数 (23 位)

符号位

0	1	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	1	1	1	1	0	1	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

阶码 (8 位)

移码表示

偏置值: $2^{8-1} = 128$

$20 + 128 = 148 = 1001\ 0100\text{B}$

浮点数编码：规格化

机器字长 = 32 位

规格化浮点数的标志是尾数部分的最高位是 1

0.0000 0000 1000 0111 1011 $\times 2^{16}$

0.0000 0000 0000 1000 0111 1011 $\times 2^{20}$

规格化操作

0.1000 0111 1011 $\times 2^8$

符号位

尾数 (23 位)

0	1	0	0	0	1	0	0	0	1	0	0	0	0	1	1	1	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

阶码 (8 位)

浮点数编码：规格化

规格化浮点数的标志是尾数部分的最高位是 1

左规

0.00 1000 0111 1011 $\times 2^{10}$



0.0 1000 0111 1011 $\times 2^9$



0.1000 0111 1011 $\times 2^8$

右规

10.00 0111 1011 $\times 2^6$



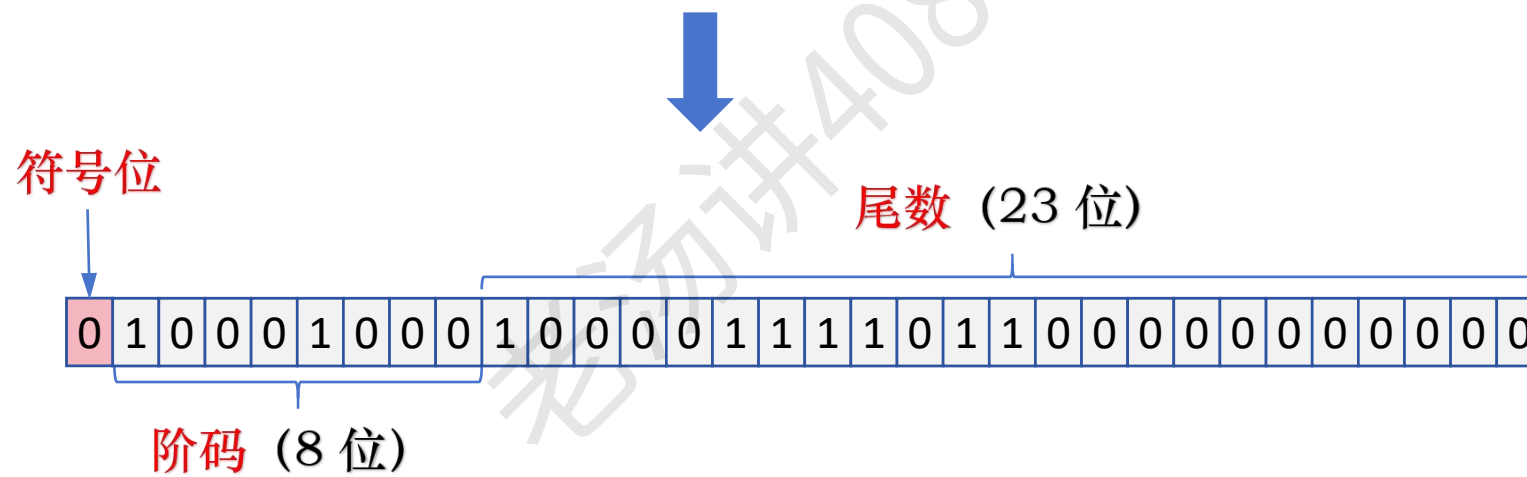
1.000 0111 1011 $\times 2^7$



0.1000 0111 1011 $\times 2^8$

浮点数编码标准

$$0.1000\ 0111\ 1011 \times 2^8$$

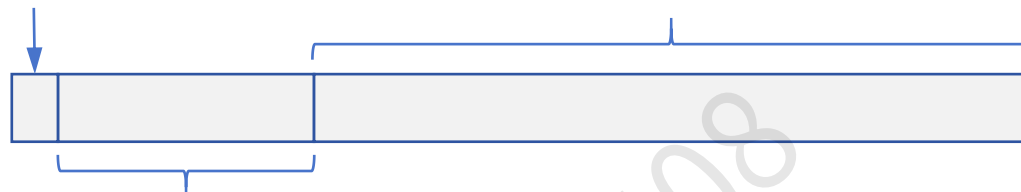


IEEE 754 浮点数标准

32 位单精度格式

符号 s (1 位)

尾数 f (23 位) 原码表示

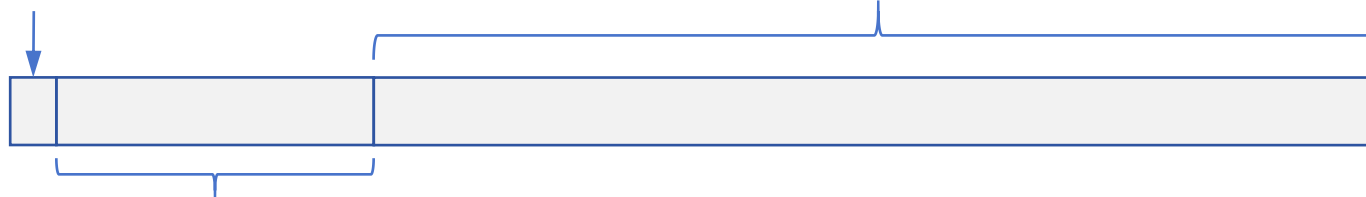


阶码 e (8 位) 移码表示 偏置值 = $2^{8-1} - 1 = 127$

64 位双精度格式

符号 s (1 位)

尾数 f (52 位) 原码表示



阶码 e (11 位) 移码表示 偏置值 = $2^{11-1} - 1 = 1023$





IEEE 754 规定尾数小数点前一位总是 1



单精度格式的 23 位尾数实际上表示了 24 位有效数字

IEEE 754 浮点数标准

$$-0.1000\ 0111\ 1011\ 1101\ 1110\ 0011 \times 2^{16}$$



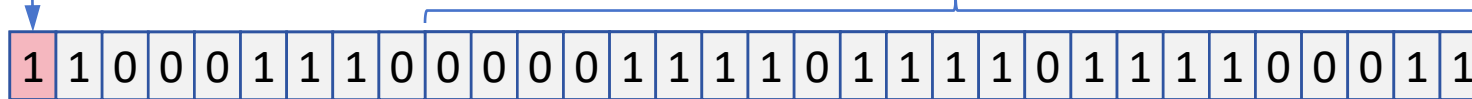
$$-1.000\ 0111\ 1011\ 1101\ 1110\ 0011 \times 2^{15}$$

隐藏位

IEEE 754 规定尾数小数点前一位总是 1

符号 s (1 位)

尾数 f (23 位)



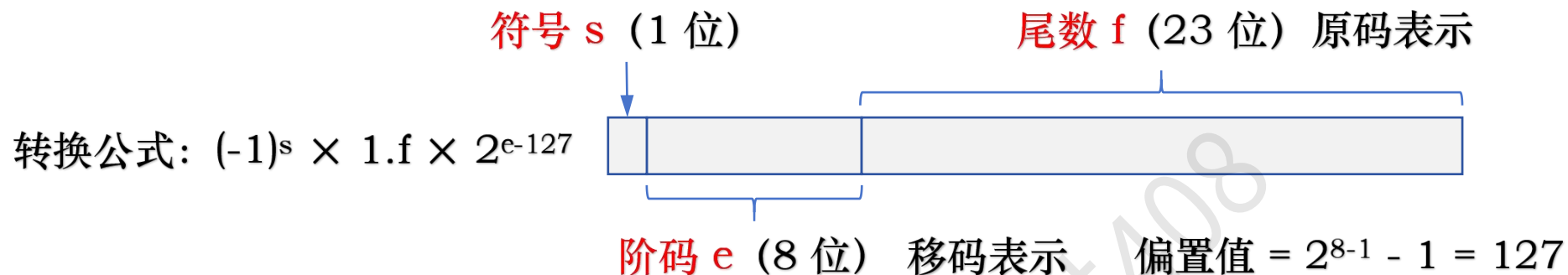
阶码 e (8 位)

$$\text{偏置值} = 2^{8-1} - 1 = 127 \quad 15 + 127 = 1111\text{B} + 1000\ 0000\text{B} - 1 = 1000\ 1110\text{B}$$

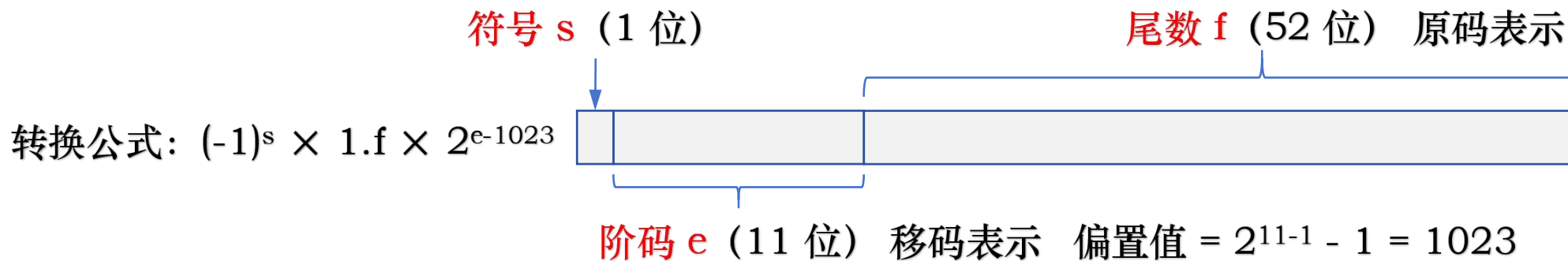
IEEE 754 浮点数标准

IEEE 754 浮点数标准

32 位单精度格式



64 位双精度格式



IEEE 754 浮点数标准：全 0 阶码全 0 尾数 (+0 / -0)

	32 位单精度格式				64 位双精度格式			
值的类型	符号 s	阶码 e	尾数 f	值	符号 s	阶码 e	尾数 f	值
正零	0	0	0	+0	0	0	0	+0
负零	1	0	0	-0	1	0	0	-0

符号 s (1 位)

尾数 f (23 位)



阶码 e (8 位)

IEEE 754 浮点数标准：全 1 阶码全 0 尾数 ($+\infty$ / $-\infty$)

	32 位单精度格式				64 位双精度格式			
值的类型	符号 s	阶码 e	尾数 f	值	符号 s	阶码 e	尾数 f	值
正零	0	0	0	$+0$	0	0	0	$+0$
负零	1	0	0	-0	1	0	0	-0
正无穷大	0	255 (全 1)	0	$+\infty$	0	2047 (全 1)	0	$+\infty$
负无穷大	1	255 (全 1)	0	$-\infty$	1	2047 (全 1)	0	$-\infty$



$+\infty$ 在数值上大于所有有限值 $-\infty$ 在数值上小于所有有限数

IEEE 754 浮点数标准：全 1 阶码非 0 尾数 (NaN)

	32 位单精度格式				64 位双精度格式			
值的类型	符号 s	阶码 e	尾数 f	值	符号 s	阶码 e	尾数 f	值
正零	0	0	0	+0	0	0	0	+0
负零	1	0	0	-0	1	0	0	-0
正无穷大	0	255 (全 1)	0	$+\infty$	0	2047 (全 1)	0	$+\infty$
负无穷大	1	255 (全 1)	0	$-\infty$	1	2047 (全 1)	0	$-\infty$
无定义数	0 或 1	255 (全 1)	$\neq 0$	NaN	0 或 1	2047 (全 1)	$\neq 0$	NaN

符号 s (1 位)

尾数 f (23 位)



阶码 e (8 位)

- $(+\infty) + (-\infty)$ $0 \times \infty$
- $(+\infty) - (+\infty)$ $0 / 0$
- $(-\infty) + (+\infty)$
- $(-\infty) - (-\infty)$

NaN: Not a Number, 表示一个没有定义的数，称为非数

IEEE 754 浮点数标准：阶码非全 0 且非全 1（规格化非 0 数）

	32 位单精度格式				64 位双精度格式			
值的类型	符号 s	阶码 e	尾数 f	值	符号 s	阶码 e	尾数 f	值
正零	0	0	0	+0	0	0	0	+0
负零	1	0	0	-0	1	0	0	-0
正无穷大	0	255 (全 1)	0	$+\infty$	0	2047 (全 1)	0	$+\infty$
负无穷大	1	255 (全 1)	0	$-\infty$	1	2047 (全 1)	0	$-\infty$
无定义数	0 或 1	255 (全 1)	$\neq 0$	NaN	0 或 1	2047 (全 1)	$\neq 0$	NaN
规格化非零正数	0	$0 < e < 255$	f	$1.f \times 2^{e-127}$	0	$0 < e < 2047$	f	$1.f \times 2^{e-1023}$
规格化非零负数	1	$0 < e < 255$	f	$-1.f \times 2^{e-127}$	1	$0 < e < 2047$	f	$-1.f \times 2^{e-1023}$

单精度指数范围：-126 ~ +127

双精度指数范围：-1022 ~ +1023

IEEE 754 浮点数标准：32 位单精度表示范围

单精度指数范围：-126 ~ +127

尾数位数：23

最大非规格化负数

最小非规格化正数

$$\begin{array}{r} 1.1...11 \\ + 0.0...01 \times 2^{-23} \\ \hline 10.0...00 \times 2^{-23} \end{array}$$

$$-2^{-149}$$

$$2^{-149}$$

$$-2^{-23} \times 2^{-126}$$

$$2^{-23} \times 2^{-126}$$

$$-0.0...01 \times 2^{-126}$$

零

$$0.0...01 \times 2^{-126}$$

规格化
负数

非规格化
负数

非规格化
正数

规格化
正数

$$-1.1...11 \times 2^{127} \quad -1.0...00 \times 2^{126}$$

$$1.0...00 \times 2^{126} \quad 1.1...11 \times 2^{127}$$

$$-(2 - 2^{-23}) \times 2^{127}$$

$$-2^{-126}$$

$$2^{-126}$$

$$(2 - 2^{-23}) \times 2^{127}$$

$$-(2^{128} - 2^{104})$$

最大规格化负数

最小规格化正数

$$(2^{128} - 2^{104})$$

最小规格化负数

最大规格化正数

IEEE 754 浮点数标准：全 0 阶码非 0 尾数（非规格化数）

	32 位单精度格式				64 位双精度格式			
值的类型	符号 s	阶码 e	尾数 f	值	符号 s	阶码 e	尾数 f	值
正零	0	0	0	+0	0	0	0	+0
负零	1	0	0	-0	1	0	0	-0
正无穷大	0	255 (全 1)	0	$+\infty$	0	2047 (全 1)	0	$+\infty$
负无穷大	1	255 (全 1)	0	$-\infty$	1	2047 (全 1)	0	$-\infty$
无定义数	0 或 1	255 (全 1)	$\neq 0$	NaN	0 或 1	2047 (全 1)	$\neq 0$	NaN
规格化非零正数	0	$0 < e < 255$	f	$1.f \times 2^{e-127}$	0	$0 < e < 2047$	f	$1.f \times 2^{e-1023}$
规格化非零负数	1	$0 < e < 255$	f	$-1.f \times 2^{e-127}$	1	$0 < e < 2047$	f	$-1.f \times 2^{e-1023}$
非规格化正数	0	0	$f \neq 0$	$0.f \times 2^{-126}$	0	0	$f \neq 0$	$0.f \times 2^{-1022}$
非规格化负数	1	0	$f \neq 0$	$-0.f \times 2^{-126}$	1	0	$f \neq 0$	$-0.f \times 2^{-1022}$

非规格化数的特点：

- ① 阶码全 0
- ② 尾数不为 0
- ③ 指数是 -126 ④ 隐藏位是 0 尾数部分的变化范围是：0.00...1 ~ 0.11...1

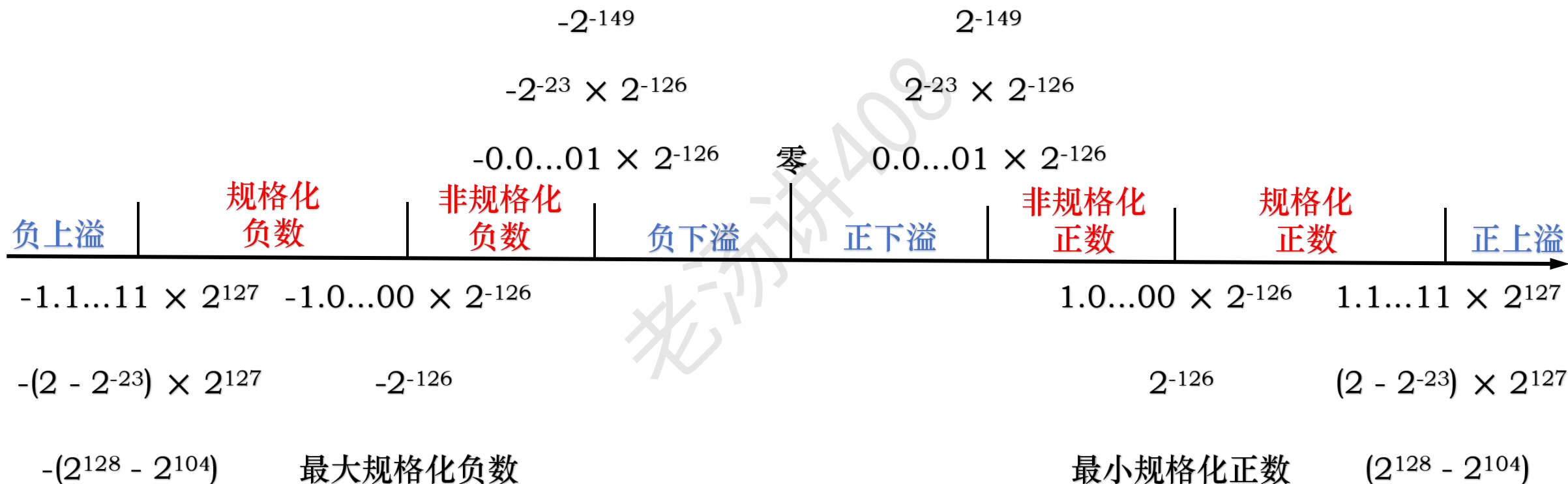
IEEE 754 浮点数标准：32 位单精度表示范围

单精度指数范围：-126 ~ +127

尾数位数：23

最大非规格化负数

最小非规格化正数



最小规格化负数

最大规格化正数

数据一旦产生上溢，计算机必须中断运算操作，进行溢出处理

数据下溢时，浮点数值趋于零，计算机将其当作机器零处理

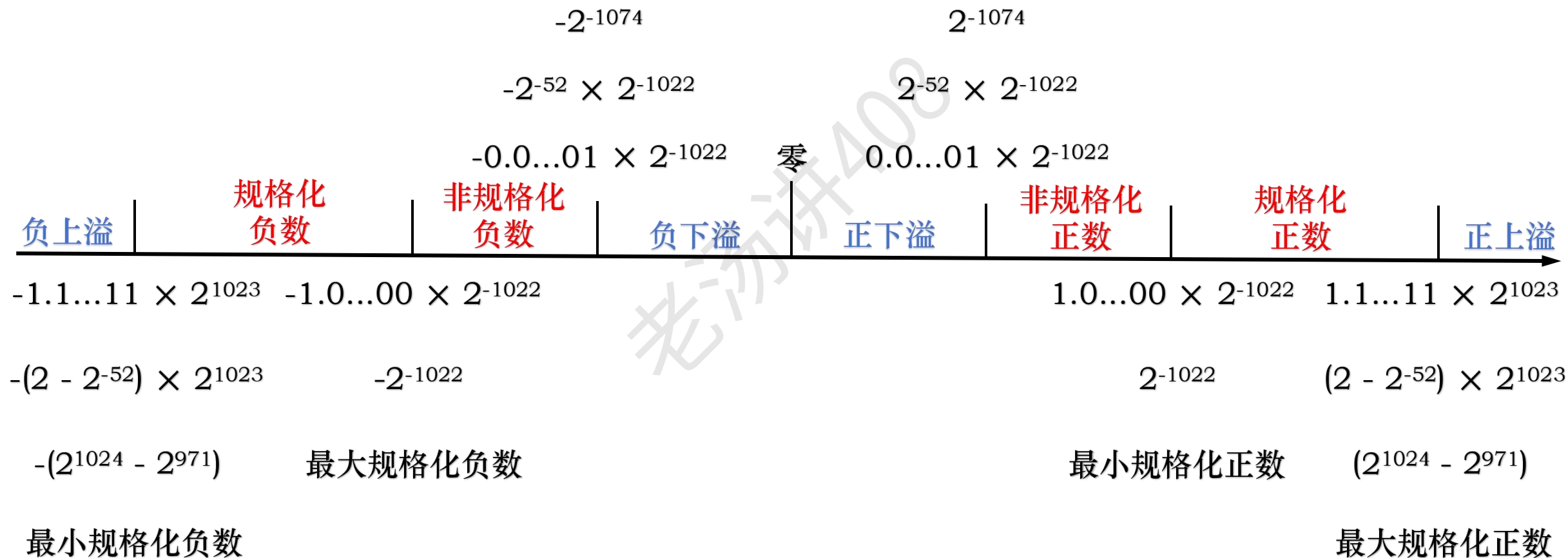
IEEE 754 浮点数标准：64 位双精度表示范围

双精度指数范围：-1022 ~ +1023

尾数位数：52

最大非规格化负数

最小非规格化正数



IEEE 754 浮点数标准：特殊值

	32 位单精度格式				64 位双精度格式			
值的类型	符号 s	阶码 e	尾数 f	值	符号 s	阶码 e	尾数 f	值
正零	0	0	0	+0	0	0	0	+0
负零	1	0	0	-0	1	0	0	-0
正无穷大	0	255 (全 1)	0	$+\infty$	0	2047 (全 1)	0	$+\infty$
负无穷大	1	255 (全 1)	0	$-\infty$	1	2047 (全 1)	0	$-\infty$
无定义数	0 或 1	255 (全 1)	$\neq 0$	NaN	0 或 1	2047 (全 1)	$\neq 0$	NaN
规格化非零正数	0	$0 < e < 255$	f	$1.f \times 2^{e-127}$	0	$0 < e < 2047$	f	$1.f \times 2^{e-1023}$
规格化非零负数	1	$0 < e < 255$	f	$-1.f \times 2^{e-127}$	1	$0 < e < 2047$	f	$-1.f \times 2^{e-1023}$
非规格化正数	0	0	$f \neq 0$	$0.f \times 2^{-126}$	0	0	$f \neq 0$	$0.f \times 2^{-1022}$
非规格化负数	1	0	$f \neq 0$	$-0.f \times 2^{-126}$	1	0	$f \neq 0$	$-0.f \times 2^{-1022}$

IEEE 754 浮点数标准：C 语言中的浮点数类型

float → 32 位单精度浮点数

double → 64 位双精度浮点数

符号 s (1 位)

尾数 f (23 位)



阶码 e (8 位)

float $f = -5.0;$

IEEE 754 浮点数标准: int、float、double 之间的强制类型转换

int 表示整数范围 $-2^{31} \sim 2^{31}-1$

① int -> float

不会发生溢出

float 表示范围 $-(2 - 2^{-23}) \times 2^{127} \sim (2 - 2^{-23}) \times 2^{127}$

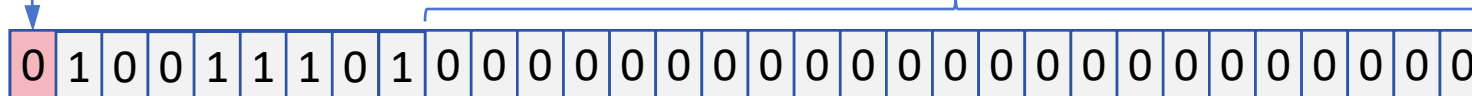
但有可能有数据被舍入

```
int a = 1073741825; // 0100 0000 0000 0000 0000 0000 0000 0001
float f = a;
printf("%f\n", f); // 1073741824.000000
```

1.00 0000 0000 0000 0000 0000 0000 0001 $\times 2^{30}$

符号位

尾数 (23 位)



阶码 (8 位)

float 表示的范围比 int 要大, 但是 float 的精度没有 int 高

IEEE 754 浮点数标准：int、float、double 之间的强制类型转换

② int 或 float 转换成 double

int 表示整数范围 $-2^{31} \sim 2^{31}-1$

float 表示范围 $-(2 - 2^{-23}) \times 2^{127} \sim (2 - 2^{-23}) \times 2^{127}$

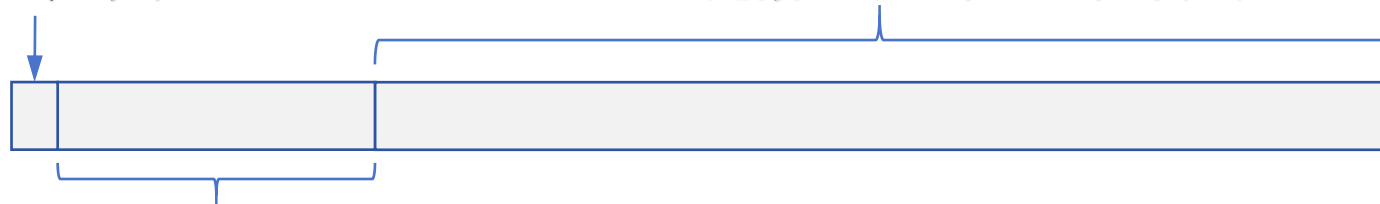
不会发生溢出

double 表示范围 $-(2 - 2^{-52}) \times 2^{1023} \sim (2 - 2^{-52}) \times 2^{1023}$

数据不会被舍掉

符号 **s** (1 位)

尾数 **f** (52 位) 原码表示



阶码 **e** (11 位) 移码表示 偏置值 = $2^{11-1} - 1 = 1023$

IEEE 754 浮点数标准：int、float、double 之间的强制类型转换

③ double 转换成 float

float 表示范围 $-(2 - 2^{-23}) \times 2^{127} \sim (2 - 2^{-23}) \times 2^{127}$

相当于: $10^{-38} \sim 10^{+38}$

double 表示范围 $-(2 - 2^{-52}) \times 2^{1023} \sim (2 - 2^{-52}) \times 2^{1023}$

相当于: $10^{-308} \sim 10^{+308}$

可能会发生溢出

```
double d = 1.3611294676837538538534984297271e+39;
```

```
float f = d;
```

`printf("%f\n", f); // inf` inf 是 infinity 的缩写，无穷大的意思

```
print float_binary(f); // 01111111100000000000000000000000
```

数据可能会被舍掉

IEEE 754 浮点数标准：int、float、double 之间的强制类型转换

④ float 或 double 转换成 int

int 表示整数范围 $-2^{31} \sim 2^{31}-1$

float 表示范围 $-(2 - 2^{-23}) \times 2^{127} \sim (2 - 2^{-23}) \times 2^{127}$

可能会发生溢出

double 表示范围 $-(2 - 2^{-52}) \times 2^{1023} \sim (2 - 2^{-52}) \times 2^{1023}$

```
float b = 10.25;  
int c = (int) b;  
printf("%d\n", c); // 10
```

小数部分直接省略

IEEE 754 浮点数标准: int、float、double 之间的强制类型转换

假定变量 i、f、d 的类型分别是 int、float 和 double，它们可以取除 $+\infty$ 、 $-\infty$ 和 NaN 以外的任意值。请判断下列每个 C 语言关系表达式在 32 位机器上运行时是否永真

① $i == (\text{int})(\text{float}) i;$ false

② $f == (\text{float})(\text{int}) f;$ false

③ $i == (\text{int})(\text{double}) i;$ true

④ $f == (\text{float})(\text{double}) f;$ true

⑤ $d == (\text{float}) d$ false

⑥ $f == -(-f)$ true

⑦ $(d + f) - d == f$ false

52 位

$$d = 1.00010..0 \times 2^{60}$$

$$f = 1.0$$
$$= 0.00000...1 \times 2^{60}$$

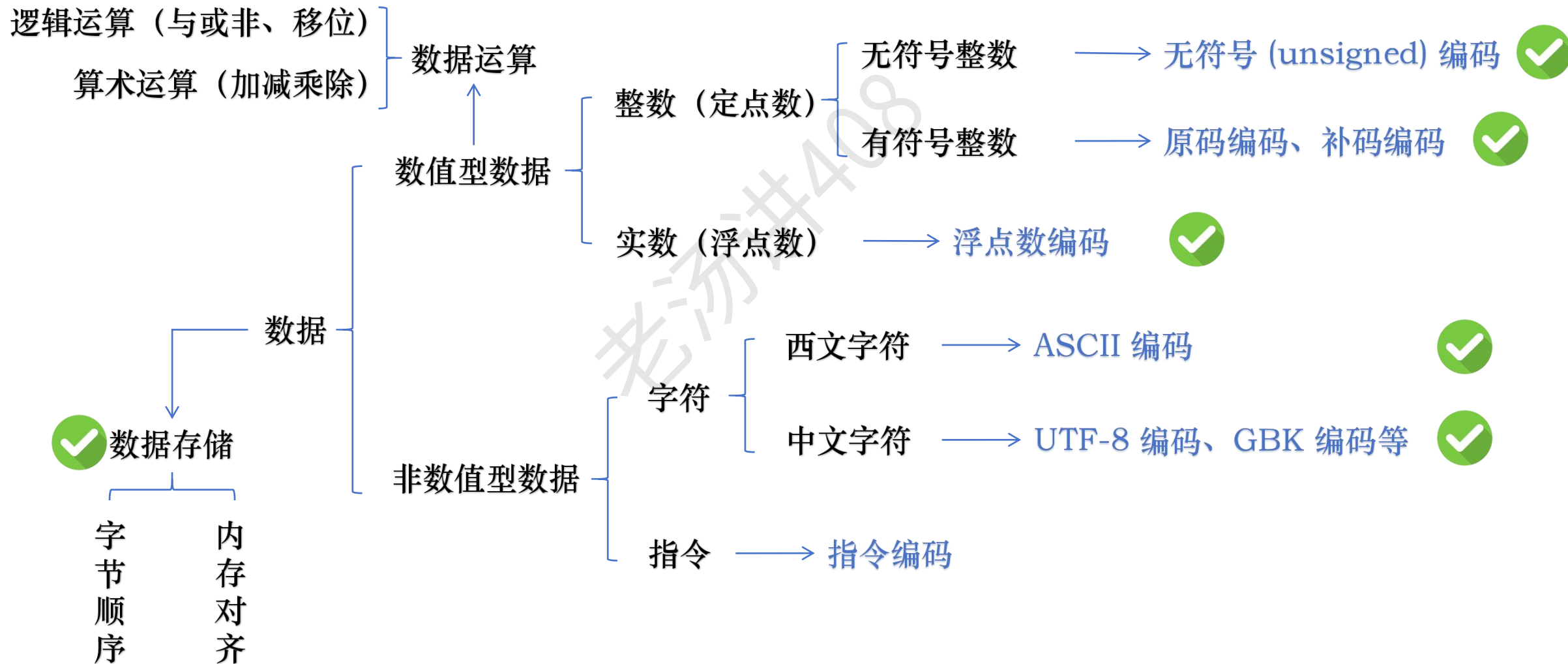
60 位

$$= 0.00000...0 \times 2^{60}$$

52 位

$d + f = d$ (大数吃小数)

数据存储：字节顺序和内存对齐



数据存储：字节顺序和内存对齐

按字节寻址

主存储器	
	...
0x100	
0x101	0100 0001
0x102	
0x103	
0x104	
0x105	
0x106	
0x107	
0x108	
0x109	
0x10A	
0x10B	
0x10C	
0x10D	
0x10E	
	...

严



11100100 10111000 10100101

① 这个数据在主存中的首地址是什么？



内存对齐

② 在主存中如何排列这 3 个字节？



字节顺序

数据存储：字节顺序

按字节寻址

主存储器

0x100

0x101

0x102

0x103

0x104

0x105

0x106

0x107

0x108

0x109

0x10A

0x10B

0x10C

0x10D

0x10E

0100 0001

最高有效字节

最低有效字节

严



11100100

10111000

10100101

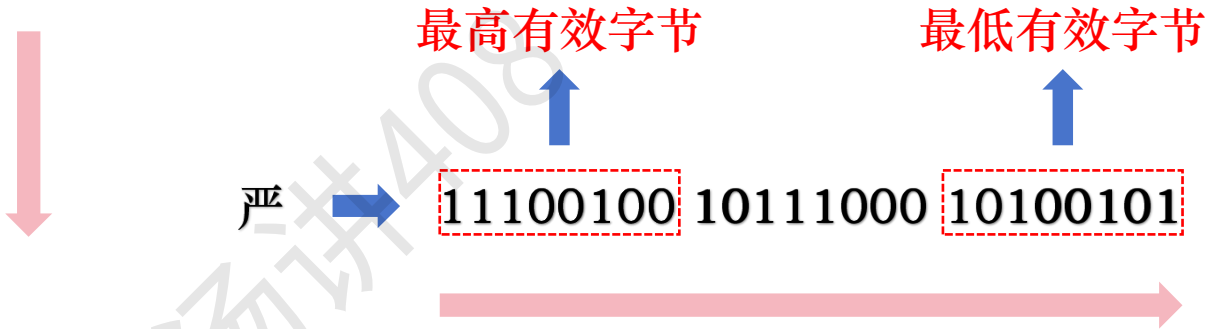


字节顺序：大端序和小端序

按字节寻址

主存储器

	...
0x100	
0x101	0100 0001
0x102	
0x103	
0x104	
0x105	
0x106	
0x107	
0x108	
0x109	
0x10A	
0x10B	
0x10C	
0x10D	
0x10E	
	...



大端序：将数据最高有效字节存放在低地址单元中，
将最低有效字节存放在高地址单元中

字节顺序：大端序和小端序

按字节寻址

主存储器

0x100

0x101

0x102

0x103

0x104

0x105

0x106

0x107

0x108

0x109

0x10A

0x10B

0x10C

0x10D

0x10E

0100 0001

最高有效字节

最低有效字节

严

11100100

10111000

10100101

大端序：将数据最高有效字节存放在低地址单元中，
将最低有效字节存放在高地址单元中

小端序：将数据最高有效字节存放在高地址单元中，
将最低有效字节存放在低地址单元中

字节顺序：大端序和小端序

IBM 360/370、Motorola 68000、MIPS、SPARC、HP PA 等机器都采用大端方式

Intel 80x86、DEC VAX 等都采用小端方式

有些微处理器芯片，如 Alpha 和 Motorola 的 PowerPC，能够运行在任意一种模式，只要在芯片加电启动时选择确定采用大端还是小端模式即可

字节顺序：大端序和小端序

按字节寻址

主存储器

0x100

0x101

0x102

0x103

0x104

0x105

0x106

0x107

0x108

0x109

0x10A

0x10B

0x10C

0x10D

0x10E

0100 0001

11100100

10111000

10100101

最高有效字节

最低有效字节

严

11100100

10111000

10100101

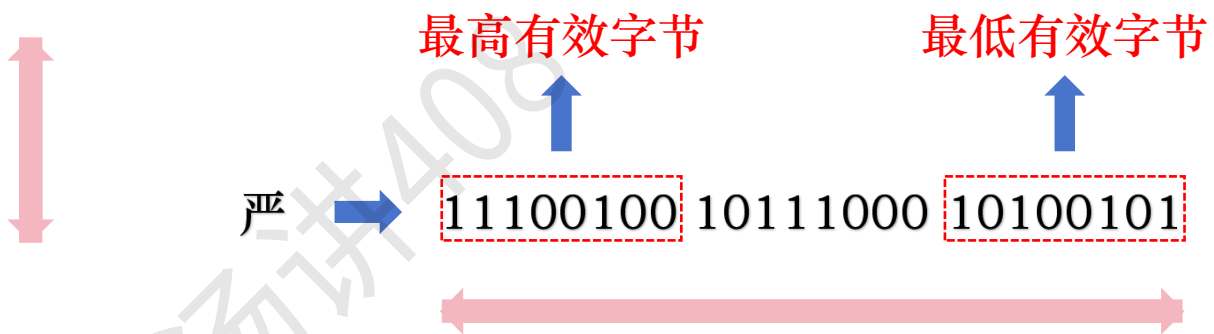
大端序：将数据最高有效字节存放在低地址单元中，
将最低有效字节存放在高地址单元中

字节顺序：大端序和小端序

按字节寻址

主存储器

	...
0x100	
0x101	0100 0001
0x102	
0x103	
0x104	10100101
0x105	10111000
0x106	11100100
0x107	
0x108	
0x109	
0x10A	
0x10B	
0x10C	
0x10D	
0x10E	
	...



小端序：将数据最高有效字节存放在高地址单元中，
将最低有效字节存放在低地址单元中

字节顺序：大端序和小端序

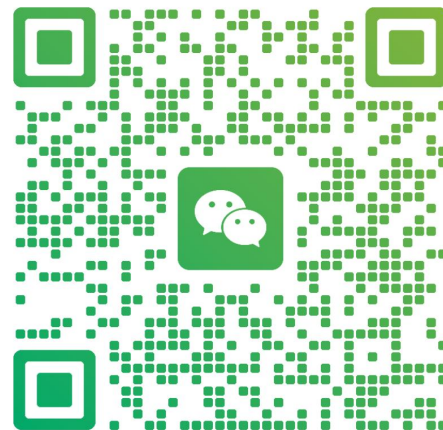
加老汤微信，帮你 408 冲 120+
咨询【老汤 408 高分特训营】



网络字节序
大端序传输

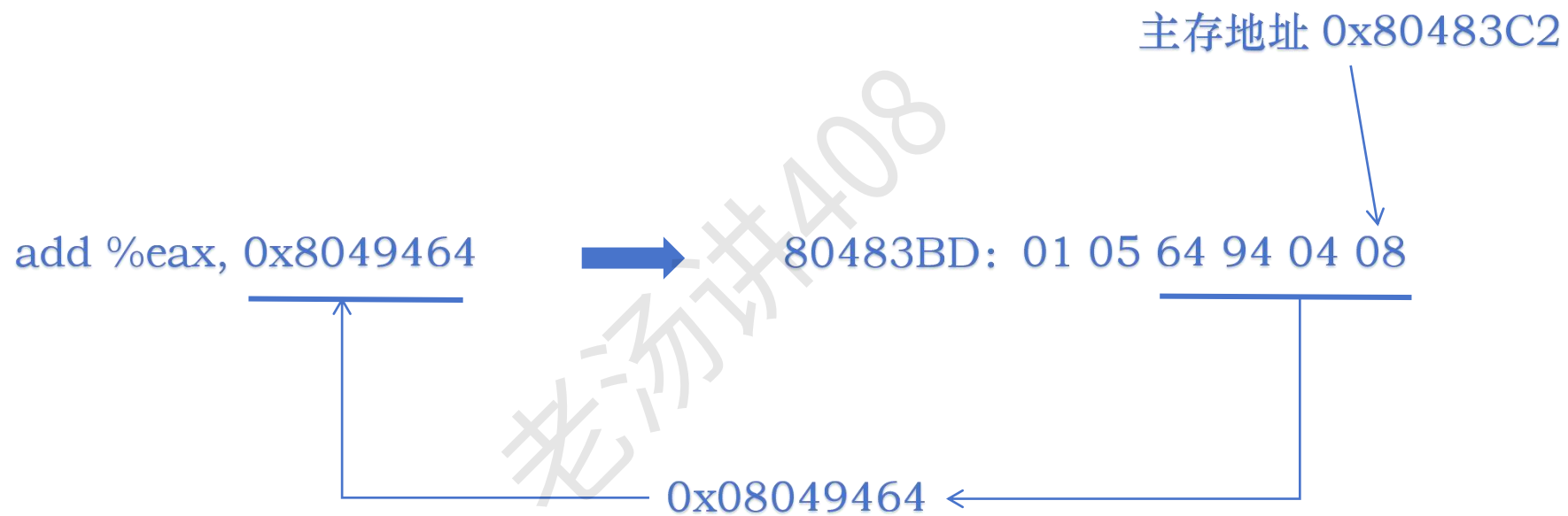


老汤
浙江 杭州



扫一扫上面的二维码图案，加我为朋友。

字节顺序：大端序和小端序



小端序存储

数据存储：内存对齐

主存地址 主存储器

0	
1	
✓ 2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	
15	
.....	...

int a = -456789



补码

11111111 11111001 00000111 10101011

① 这个数据在主存中的首地址是什么？

② 在主存中如何排列这 4 个字节？

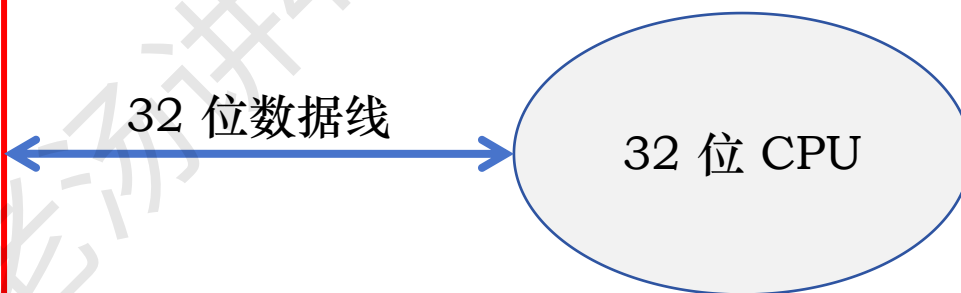


大端序

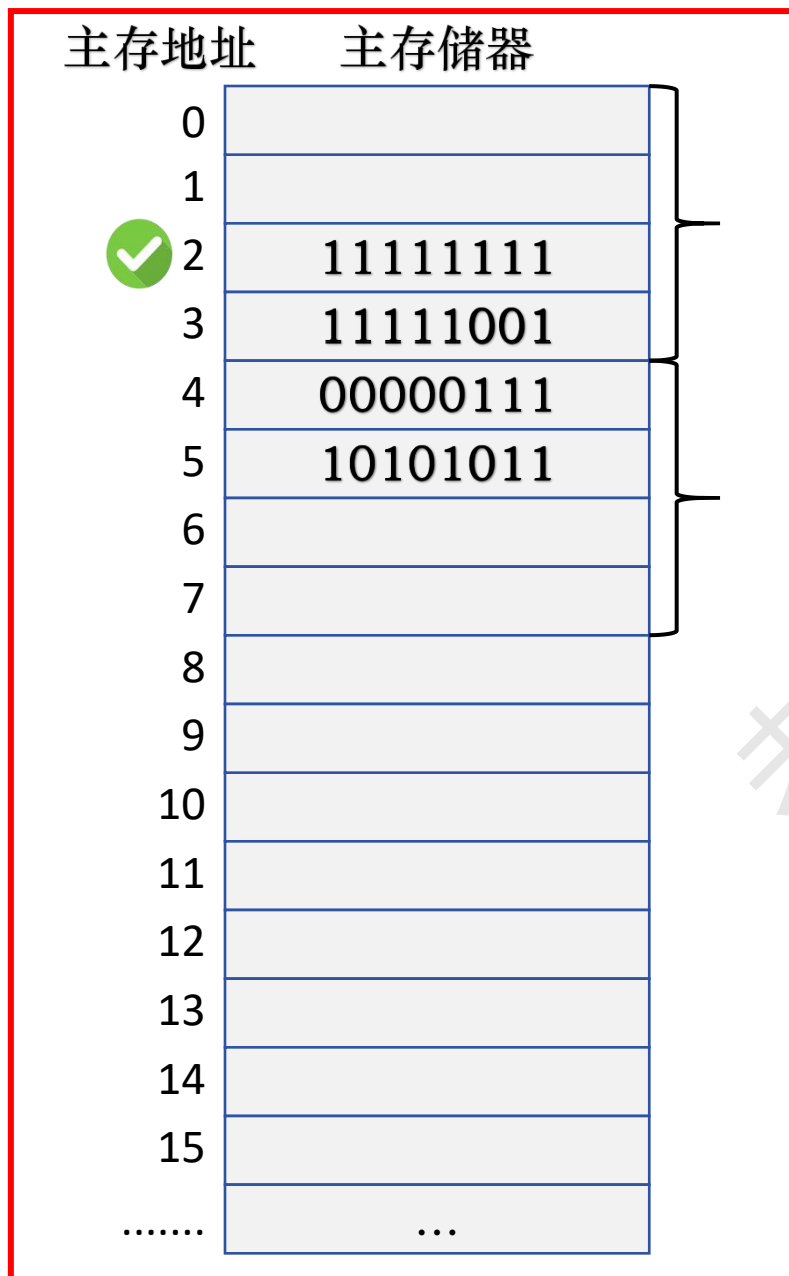
数据存储：内存对齐

主存地址	主存储器
0	✓
1	
2	11111111
3	11111001
4	000✓111
5	10101011
6	
7	
8	✓
9	
10	
11	
12	✓
13	
14	
15	
.....	...

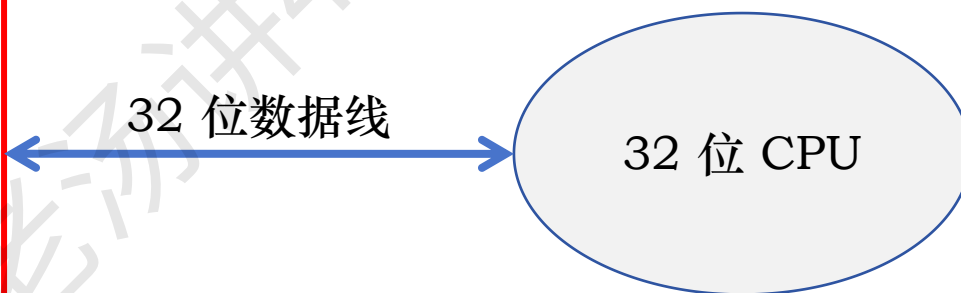
从硬件角度看：数据线的 32 根线路直接将内存中连续的 32 位（4 字节）数据位对位地传输到 CPU 内部的寄存器等部件中，不需要额外的处理来拼接或拆分数据



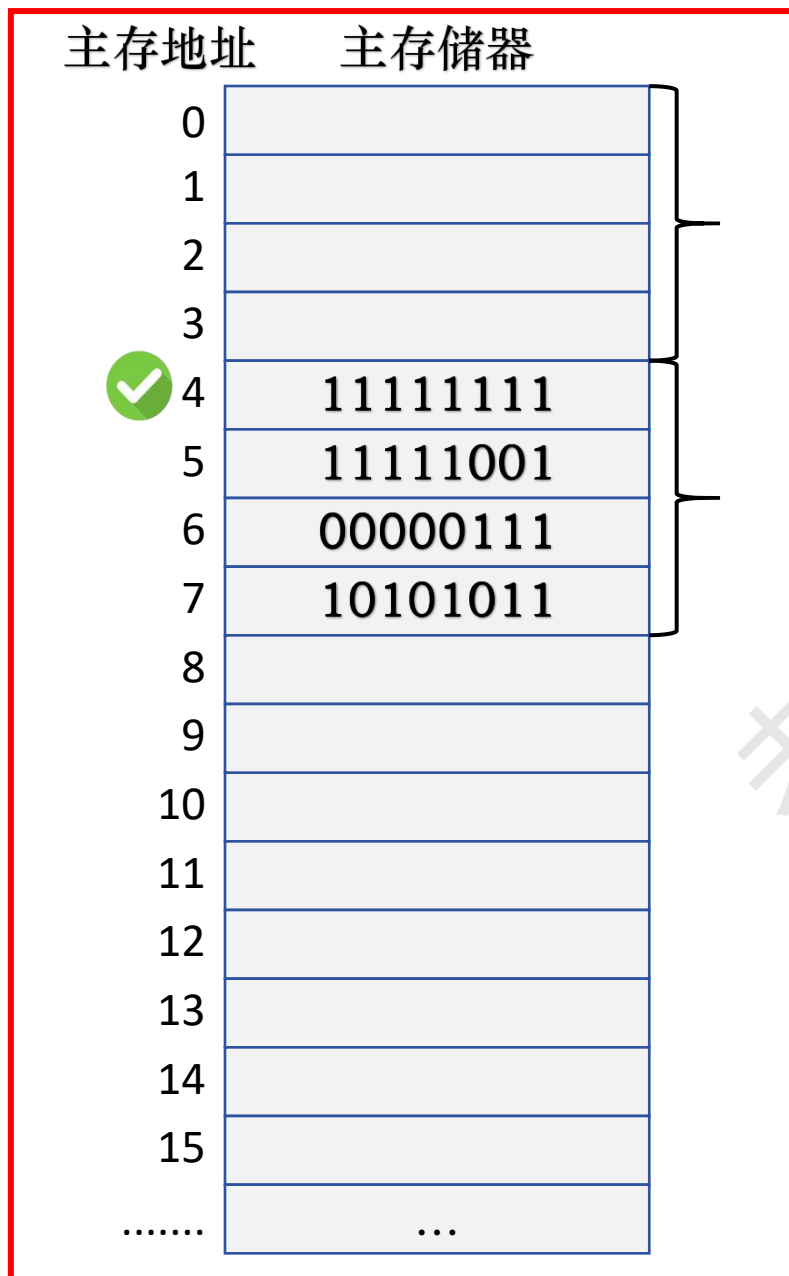
数据存储：内存对齐



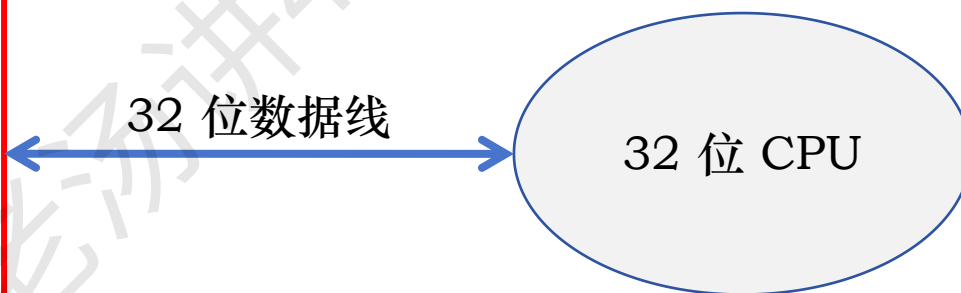
如果要读取的数据首地址不是 4 的倍数的话，就需要额外的出来来拼接或拆分数据



数据存储：内存对齐

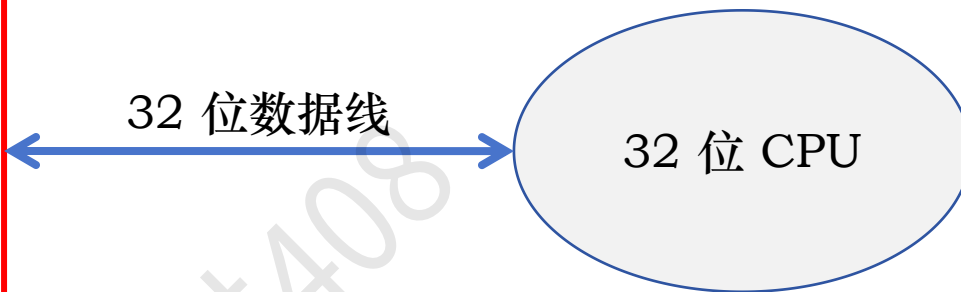


如果要读取的数据首地址不是 4 的倍数的话，就需要额外的出来来拼接或拆分数据



数据存储：内存对齐

主存地址	主存储器
0	00000000
1	00001001
2	
3	
4	11111111
5	11111001
6	00000111
7	10101011
8	
9	
10	
11	
12	
13	
14	
15	
.....	...

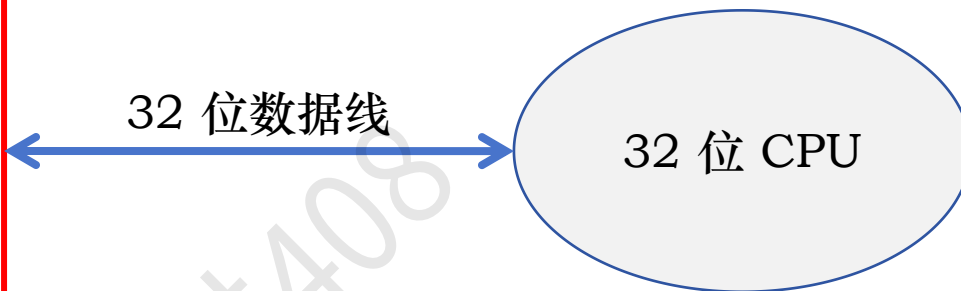


```
struct person {  
    unsigned short age;  
    int id;  
};  
  
int main() {  
    printf("%d\n", sizeof(struct person));  
    return 1;  
};
```

数据存储：内存对齐

规则一：每个成员存储的起始地址 % 该成员的长度 = 0

主存地址	主存储器
0	00101010
1	
2	00000000
3	00001001
4	11111111
5	11111001
6	00000111
7	10101011
8	
9	
10	
11	
12	
13	
14	
15	
.....	...

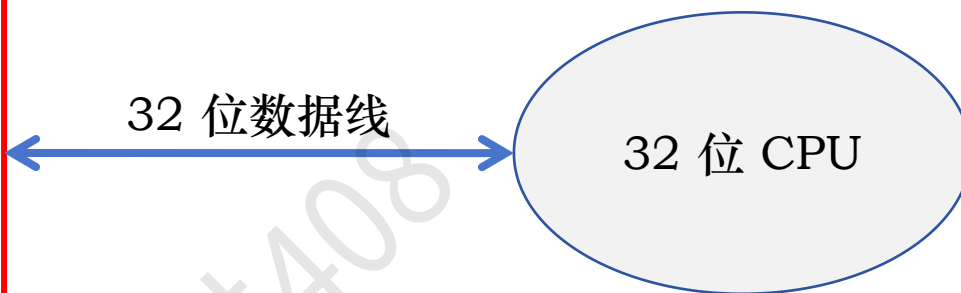
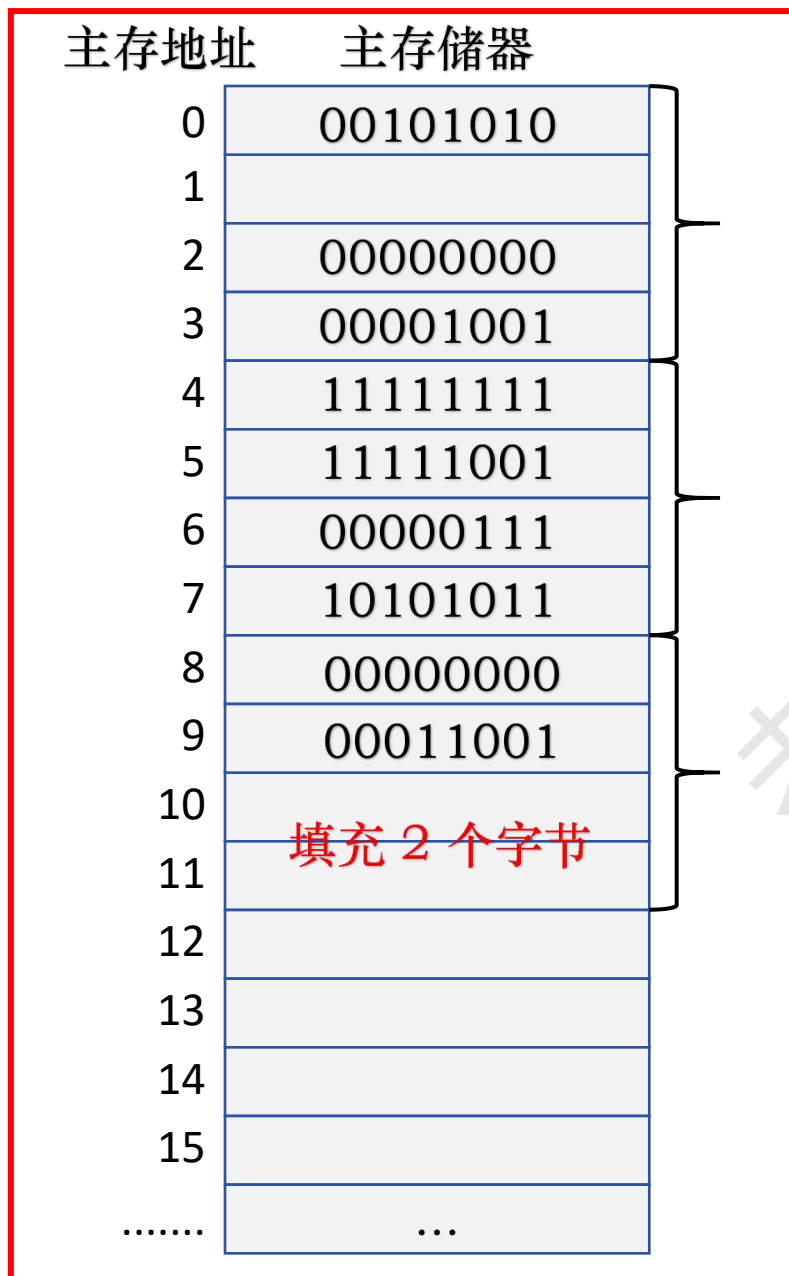


```
struct person {  
    char flag;  
    unsigned short age;  
    int id;  
};  
  
int main() {  
    printf("%d\n", sizeof(struct person));  
    return 1;  
};
```

数据存储：内存对齐

规则一：每个成员存储的起始地址 % 该成员的长度 = 0

规则二：结构体的长度也必须是最大成员长度的整数倍



```
struct person {  
    char flag;  
    unsigned short age;  
    int id;  
    unsigned short height;  
};
```

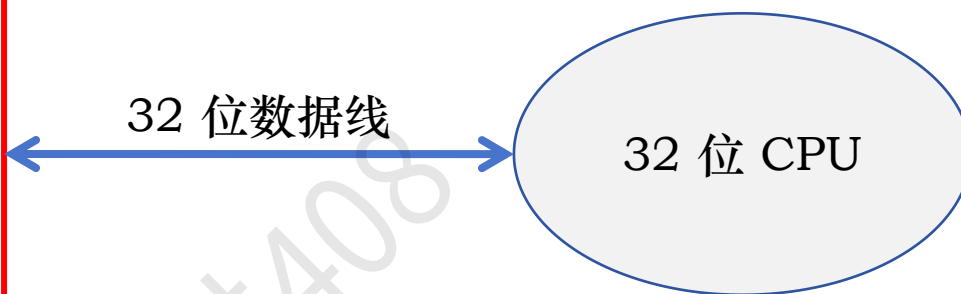
```
int main() {  
    printf("%d\n", sizeof(struct person));  
    return 1;  
};
```

数据存储：内存对齐

主存地址	主存储器
0	00101010
1	
2	00000000
3	00001001
4	11111111
5	11111001
6	00000111
7	10101011
8	00000000
9	00011001
10	填充 2 个字节
11	
12	
13	
14	
15	
.....	...

规则一：每个成员存储的起始地址 % 该成员的长度 = 0

规则二：结构体的长度也必须是最大成员长度的整数倍



空间换时间

```
struct person {  
    char flag;  
    unsigned short age;  
    int id;  
    unsigned short height;  
};
```

```
int main() {  
    printf("%d\n", sizeof(struct person));  
    return 1;  
};
```